

Hình 10-11. Đường cong học tập: tổn thất huấn luyện trung bình và độ chính xác được đo lường qua từng kỳ nguyên, và tổn thất xác thực trung bình và độ chính xác được đo lường vào cuối mỗi kỳ nguyên.

Bạn có thể thấy rằng cả độ chính xác huấn luyện và độ chính xác kiểm định đều tăng đều đặn trong quá trình huấn luyện, trong khi tổn thất huấn luyện và tổn thất kiểm định giảm. Tốt! Các đường cong kiểm định ban đầu khá gần nhau, nhưng chúng ngày càng cách xa nhau hơn theo thời gian, điều này cho thấy có một chút hiện tượng quá khớp. Trong trường hợp cụ thể này, mô hình dường như hoạt động tốt hơn trên tập dữ liệu kiểm định so với tập dữ liệu huấn luyện ở giai đoạn đầu huấn luyện. Nhưng thực tế không phải vậy: thực tế, lỗi kiểm định được tính toán vào cuối mỗi epoch, trong khi lỗi huấn luyện được tính toán bằng cách sử dụng giá trị trung bình động trong mỗi epoch. Vì vậy, đường cong huấn luyện cần được dịch chuyển sang trái nửa epoch. Nếu bạn làm như vậy, bạn sẽ thấy rằng các đường cong huấn luyện và kiểm định gần như trùng khớp hoàn hảo ở giai đoạn đầu huấn luyện.

Hiệu năng trên tập huấn luyện thường vượt trội hơn hiệu năng trên tập kiểm chứng, điều này thường xảy ra khi bạn huấn luyện đủ lâu. Bạn có thể nhận thấy rằng mô hình chưa hoàn toàn hội tụ, vì tổn thất trên tập kiểm chứng vẫn đang giảm, vì vậy bạn nên tiếp tục huấn luyện. Chỉ cần gọi lại phương thức `fit()`, vì Keras sẽ tiếp tục huấn luyện từ nơi nó đã dừng lại: bạn sẽ đạt được độ chính xác trên tập kiểm chứng khoảng 89,8%, trong khi độ chính xác trên tập huấn luyện sẽ tiếp tục tăng lên đến 100% (điều này không phải lúc nào cũng đúng).

Nếu bạn không hài lòng với hiệu suất của mô hình, bạn nên quay lại và tinh chỉnh các siêu tham số. Đầu tiên cần kiểm tra là tốc độ học (learning rate). Nếu điều đó không giúp ích, hãy thử một trình tối ưu hóa khác (và luôn luôn tinh chỉnh lại tốc độ học sau khi thay đổi bất kỳ siêu tham số nào). Nếu hiệu suất vẫn chưa tốt, hãy thử tinh chỉnh các siêu tham số của mô hình như số lớp, số nơ-ron trên mỗi lớp và loại hàm kích hoạt được sử dụng cho mỗi lớp ẩn. Bạn cũng có thể thử tinh chỉnh các siêu tham số khác, chẳng hạn như kích thước lô (batch size) (có thể được thiết lập trong phương thức `fit()` bằng cách sử dụng đối số `batch_size`, mặc định là 32). Chúng ta sẽ quay lại việc tinh chỉnh siêu tham số ở cuối chương này. Khi bạn hài lòng với độ chính xác xác thực của mô hình, bạn nên đánh giá nó trên tập dữ liệu kiểm thử để ước tính lỗi tổng quát trước khi triển khai mô hình.

sản xuất. Bạn có thể dễ dàng thực hiện điều này bằng cách sử dụng phương thức `evaluate()` (nó cũng hỗ trợ một số đối số khác, chẳng hạn như `batch_size` và `sample_weight`; vui lòng xem tài liệu để biết thêm chi tiết):

```
>>> model.evaluate(X_test, y_test)
313/313 [=====] - 0 giây 626 micro giây/bước
- tổn thất: 0.3243 - độ chính xác phân loại thưa: 0.8864
[0.32431697845458984, 0.8863999843597412]
```

Như chúng ta đã thấy trong **Chương 2**, việc đạt được hiệu suất thấp hơn một chút là điều thường thấy. tập dữ liệu kiểm thử cho kết quả tốt hơn tập dữ liệu xác thực, vì các siêu tham số đã được điều chỉnh. trên tập dữ liệu xác thực, chứ không phải tập dữ liệu kiểm thử (tuy nhiên, trong ví dụ này, chúng ta không làm vậy). (Tôi không tinh chỉnh bất kỳ siêu tham số nào, nên độ chính xác thấp hơn chỉ là do không may mắn thôi). Hãy nhớ đừng sa vào cám dỗ chỉnh sửa các siêu tham số trong quá trình kiểm thử. thiết lập, nếu không, ước tính của bạn về sai số tổng quát sẽ quá lạc quan.

Sử dụng mô hình để đưa ra dự đoán

Tiếp theo, chúng ta có thể sử dụng phương thức `predict()` của mô hình để đưa ra dự đoán về các trường hợp mới. Vì chúng ta không có các trường hợp mới thực sự, nên chúng ta sẽ chỉ sử dụng Ba trường hợp đầu tiên của tập dữ liệu thử nghiệm:

```
>>> X_new = X_test[:3]
>>> y_proba = model.predict(X_new)
>>> y_proba.round(2)
mảng([[0. , 0. 0. 0.97],
      [0. , 0. , 0.99, 0. , 0.01, 0. , 0.02, 0. ,
      [0. , 1. , 0. , 0. , 0. , 0. , 0. , 0. , 0.
      ]],
      dtype=float32)
```

Như bạn thấy, với mỗi trường hợp, mô hình ước tính một xác suất cho mỗi lớp, từ lớp 0 đến lớp 9. Điều này tương tự như kết quả đầu ra của...

Phương thức `predict_proba()` trong bộ phân loại Scikit-Learn. Ví dụ, đối với

Hình ảnh đầu tiên ước tính xác suất của loại 9 (giày bốt cổ thấp) là

96%, xác suất của hạng 7 (giày thể thao) là 2%, xác suất của hạng 5

(Dép xăng đan) chiếm 1%, và xác suất của các loại khác là không đáng kể. Trong

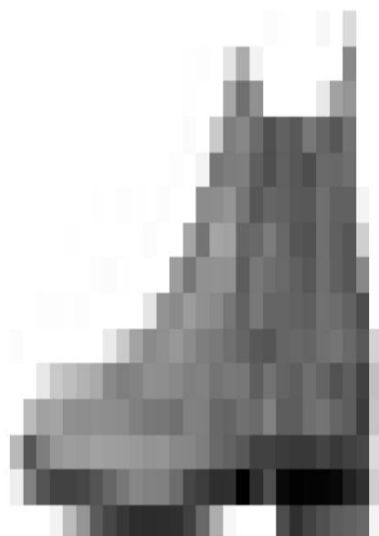
Nói cách khác, có độ tin cậy rất cao rằng hình ảnh đầu tiên là giày dép, nhiều khả năng là boot cổ ngắn nhưng cũng có thể là giày thể thao hoặc dép xăng đan. Nếu bạn chỉ quan tâm đến lớp có xác suất ước tính cao nhất (ngay cả khi xác suất đó khá thấp), thì bạn có thể sử dụng `argmax()` để lấy chỉ số lớp có xác suất cao nhất cho mỗi trường hợp:

```
>>> import numpy as np >>>
y_pred = y_proba.argmax(axis=-1) >>> y_pred
array([9,
       2, 1]) >>>
np.array(class_names)[y_pred]
array(['Ankle boot', 'Pullover', 'Trouser'], dtype='<U11')
```

Ở đây, bộ phân loại đã phân loại chính xác cả ba hình ảnh (các hình ảnh này được hiển thị trong [Hình 10-12](#)):

```
>>> y_new = y_test[:3] >>>
y_new
array([9, 2, 1], dtype=uint8)
```

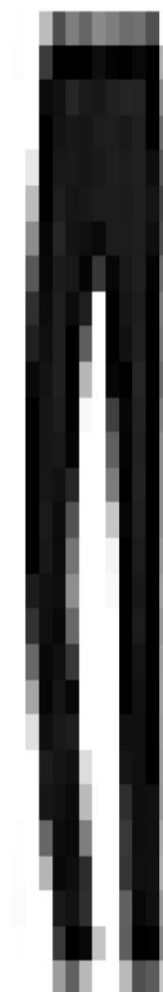
Ankle boot



Pullover



Trouser



Hình 10-12. Hình ảnh MNIST thời trang được phân loại chính xác.

Giờ bạn đã biết cách sử dụng API Sequential để xây dựng, huấn luyện và đánh giá mạng nơ-ron truyền đa lớp (MLP) phân loại. Nhưng còn hồi quy thì sao?

Xây dựng mạng nơ-ron đa lớp hồi quy (MLP) bằng API tuần tự.

Hãy quay lại với vấn đề nhà ở California và giải quyết nó bằng cách sử dụng cùng một mạng nơ-ron đa lớp (MLP) như trước, với 3 lớp ẩn, mỗi lớp gồm 50 nơ-ron, nhưng lần này được xây dựng bằng Keras.

Việc sử dụng API Sequential để xây dựng, huấn luyện, đánh giá và sử dụng mạng nơ-ron truyền đa lớp (MLP) hồi quy khá giống với những gì chúng ta đã làm cho phân loại. Sự khác biệt chính nằm ở...

Ví dụ mã sau đây cho thấy lớp đầu ra chỉ có một nơon (vì chúng ta chỉ muốn dự đoán một giá trị duy nhất) và không sử dụng hàm kích hoạt, hàm mất mát là sai số bình phương trung bình, chỉ số đánh giá là RMSE, và chúng ta đang sử dụng trình tối ưu hóa Adam giống như MLPRegressor của Scikit-Learn. Hơn nữa, trong ví dụ này, chúng ta không cần lớp Flatten, mà thay vào đó chúng ta sử dụng lớp Normalization làm lớp đầu tiên: nó thực hiện cùng một chức năng như StandardScaler của Scikit-Learn, nhưng nó phải được huấn luyện trên dữ liệu huấn luyện bằng phương thức `adapt()` trước khi gọi phương thức `fit()` của mô hình. Keras có các lớp tiền xử lý khác sẽ được đề cập trong [Chương 13](#).

```
tf.random.set_seed(42)
norm_layer =
tf.keras.layers.Normalization(input_shape=X_train.shape[1:]) model
= tf.keras.Sequential([ norm_layer,

    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(50, activation="relu"),
    tf.keras.layers.Dense(1)

]) optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=
["RootMeanSquaredError"])
norm_layer.adapt(X_train)
history = model.fit(X_train, y_train, epochs=20,
                    validation_data=(X_valid, y_valid))
mse_test, rmse_test = model.evaluate(X_test, y_test)
X_new = X_test[:3]
y_pred = model.predict(X_new)
```

GHI CHÚ

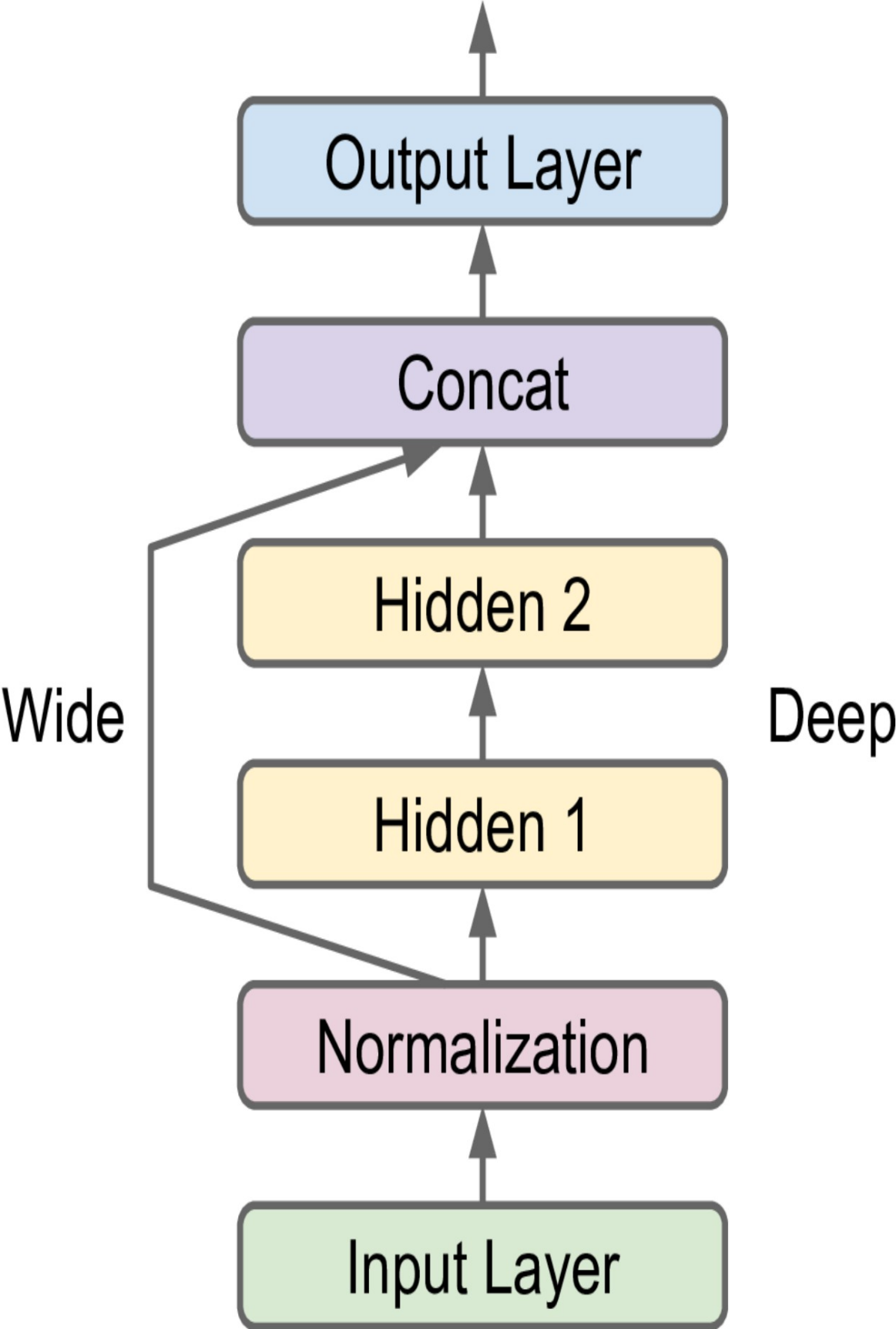
Lớp chuẩn hóa học các giá trị trung bình và độ lệch chuẩn của các đặc trưng trong dữ liệu huấn luyện khi bạn gọi phương thức `adapt()`. Tuy nhiên, khi bạn hiển thị tóm tắt của mô hình, các số liệu thống kê này được liệt kê là không thể huấn luyện được. Điều này là do các tham số này không bị ảnh hưởng bởi thuật toán Gradient Descent.

Như bạn thấy, API Sequential khá gọn gàng và dễ hiểu.

Tuy nhiên, mặc dù các mô hình tuần tự rất phổ biến, đôi khi việc xây dựng mạng nơ-ron với cấu trúc phức tạp hơn, hoặc với nhiều đầu vào hoặc đầu ra lại rất hữu ích. Để phục vụ mục đích này, Keras cung cấp API chức năng.

Xây dựng các mô hình phức tạp bằng cách sử dụng API chức năng

Một ví dụ về mạng nơ-ron phi tuần tự là mạng nơ-ron rộng và sâu (Wide & Deep neural network). Kiến trúc mạng nơ-ron này được giới thiệu trong một [bài báo năm 2016](#). Theo Heng-Tze Cheng và cộng sự, lớp đầu ¹⁵ Nó kết nối toàn bộ hoặc một phần các đầu vào trực tiếp với ra được thể hiện trong [Hình 10-13](#). Kiến trúc này cho phép mạng nơ-ron học được cả các mẫu sâu (sử dụng đường dẫn sâu) và các quy tắc đơn giản (thông qua đường dẫn ngắn). Ngược lại, một MLP thông thường buộc tất cả dữ liệu phải đi ¹⁶ qua toàn bộ chồng lớp; do đó, các mẫu đơn giản trong dữ liệu có thể bị biến dạng bởi chuỗi biến đổi này.



Hình 10-13. Mạng nơ-ron rộng và sâu

Hãy cùng xây dựng một mạng lưới thần kinh như vậy để giải quyết vấn đề nhà ở tại California:

```
normalization_layer = tf.keras.layers.Normalization()
hidden_layer1 = tf.keras.layers.Dense(30, activation="relu")
hidden_layer2 = tf.keras.layers.Dense(30, activation="relu")
concat_layer = tf.keras.layers.Concatenate()
output_layer = tf.keras.layers.Dense(1)

input_ = tf.keras.layers.Input(shape=X_train.shape[1:]) normalized
= normalization_layer(input_) hidden1 =
hidden_layer1(normalized) hidden2 =
hidden_layer2(hidden1) concat =
concat_layer([input_, hidden2]) output =
output_layer(concat)

model = tf.keras.Model(inputs=[input_], outputs=[output])
```

Nhìn chung, năm dòng đầu tiên tạo ra tất cả các lớp cần thiết để xây dựng mô hình, sáu dòng tiếp theo sử dụng các lớp này như các hàm để chuyển đổi từ đầu vào sang đầu ra, và dòng cuối cùng tạo ra một đối tượng Keras Model bằng cách trỏ đến đầu vào và đầu ra.

Hãy cùng xem xét đoạn mã này chi tiết hơn:

- Đầu tiên, chúng ta tạo ra năm lớp: một lớp Chuẩn hóa để chuẩn hóa đầu vào, hai lớp Dense với 30 nơ-ron mỗi lớp, sử dụng hàm kích hoạt ReLU, một lớp Nối chuỗi, và một lớp Dense nữa với một nơ-ron duy nhất cho lớp đầu ra, không có hàm kích hoạt nào.
- Tiếp theo, chúng ta tạo một đối tượng Input (tên biến input_ được sử dụng để tránh trùng lặp với hàm input() tích hợp sẵn của Python). Đây là thông số kỹ thuật về loại đầu vào mà mô hình sẽ nhận được, bao gồm hình dạng và tùy chọn kiểu dữ liệu, mặc định là số thực 32 bit. Một mô hình thực tế có thể có nhiều đầu vào, như chúng ta sẽ thấy ngay sau đây.
- Sau đó, chúng ta sử dụng lớp Chuẩn hóa giống như một hàm, truyền đối tượng Đầu vào cho nó. Đó là lý do tại sao nó được gọi là API Chức năng.
Lưu ý rằng chúng ta chỉ đang hướng dẫn Keras cách kết nối các lớp.

Cùng nhau; chưa có dữ liệu thực tế nào được xử lý, vì đối tượng Input chỉ là một đặc tả dữ liệu. Nói cách khác, đó là một đầu vào mang tính biểu tượng. Đầu ra của lệnh gọi này cũng mang tính biểu tượng: `normalized` không lưu trữ bất kỳ dữ liệu thực tế nào, nó chỉ được sử dụng để xây dựng mô hình.

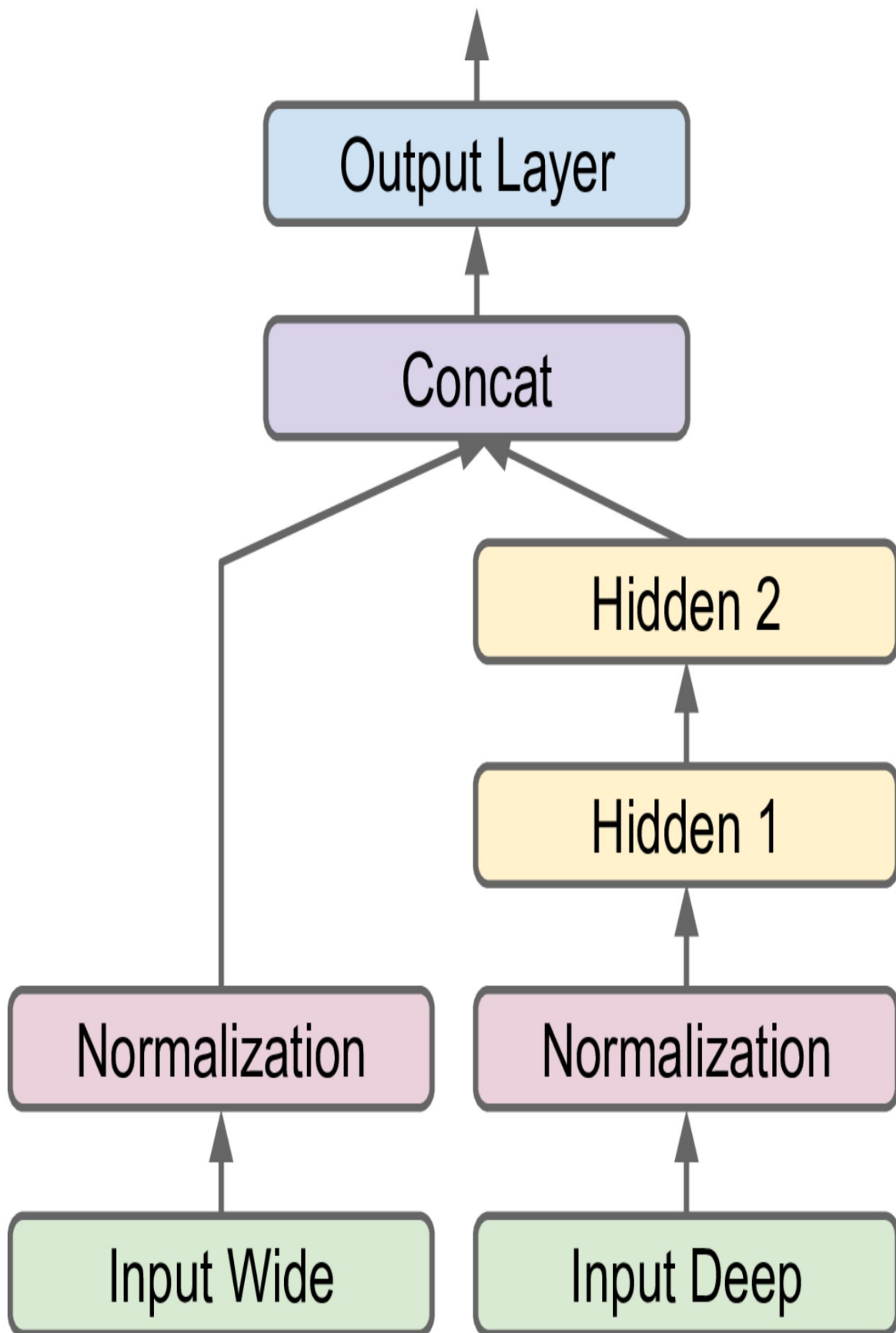
- Tương tự, chúng ta truyền giá trị đã được chuẩn hóa cho `hidden_layer1`, hàm này sẽ xuất ra `hidden1`, và chúng ta truyền `hidden1` cho `hidden_layer2`, hàm này sẽ xuất ra `hidden2`.
- Cho đến nay, chúng ta đã kết nối các lớp theo trình tự, nhưng sau đó chúng ta sử dụng `'concat_layer'` để nối đầu vào và đầu ra của lớp ẩn thứ hai. Một lần nữa, chưa có dữ liệu thực tế nào được nối: tất cả chỉ mang tính biểu tượng, để xây dựng mô hình.
- Sau đó, chúng ta truyền `concat` cho `output_layer`, hàm này sẽ cho ra kết quả cuối cùng.
- Cuối cùng, chúng ta tạo một Keras Model, chỉ định các đầu vào và đầu ra để sử dụng.

Sau khi xây dựng mô hình Keras này, mọi thứ đều giống như trước đó, vì vậy không cần phải lặp lại ở đây: bạn phải biên dịch mô hình, điều chỉnh lớp Chuẩn hóa, huấn luyện mô hình, đánh giá nó và sử dụng nó để đưa ra dự đoán.

Nhưng nếu bạn muốn gửi một tập hợp con các đặc trưng qua đường dẫn rộng và một tập hợp con khác (có thể chồng chéo) qua đường dẫn sâu (xem [Hình 10-14](#)) thì sao? Trong trường hợp này, một giải pháp là sử dụng nhiều đầu vào. Ví dụ, giả sử chúng ta muốn gửi năm đặc trưng qua đường dẫn rộng (đặc trưng từ 0 đến 4) và sáu đặc trưng qua đường dẫn sâu (đặc trưng từ 2 đến 7):

```
input_wide = tf.keras.layers.Input(shape=[5]) # các đặc trưng từ 0 đến 4
input_deep = tf.keras.layers.Input(shape=[6]) # các đặc trưng từ 2 đến 7
norm_layer_wide = tf.keras.layers.Normalization()
norm_layer_deep = tf.keras.layers.Normalization()
norm_wide = norm_layer_wide(input_wide)
norm_deep = norm_layer_deep(input_deep)
hidden1 = tf.keras.layers.Dense(30, activation="relu")(norm_deep)
hidden2 = tf.keras.layers.Dense(30, activation="relu")(hidden1)
```

```
concat = tf.keras.layers.concatenate([norm_wide, hidden2])  
output = tf.keras.layers.Dense(1)(concat)  
model = tf.keras.Model(inputs=[input_wide, input_deep], outputs=  
[output])
```



Hình 10-14. Xử lý nhiều đầu vào

Có một vài điểm cần lưu ý trong ví dụ mã này, so với ví dụ trước đó:

- Mỗi lớp Dense được tạo và gọi trên cùng một dòng. Đây là một cách làm phổ biến, vì nó giúp mã ngắn gọn hơn mà vẫn đảm bảo tính rõ ràng. Tuy nhiên, chúng ta không thể làm điều này với lớp Normalization vì chúng ta cần một tham chiếu đến lớp đó để có thể gọi phương thức `adapt()` của nó trước khi huấn luyện mô hình.
- Chúng tôi đã sử dụng `tf.keras.layers.concatenate()`, hàm này tạo ra một lớp Concatenate và gọi nó với các đầu vào được cung cấp.
- Chúng tôi đã chỉ định `inputs=[input_wide, input_deep]` khi tạo mô hình, vì có hai đầu vào.

Giờ chúng ta có thể biên dịch mô hình như bình thường, nhưng khi gọi phương thức `fit()`, thay vì truyền một ma trận đầu vào duy nhất `X_train`, chúng ta phải truyền một cặp ma trận (`X_train_wide`, `X_train_deep`): mỗi ma trận tương ứng với một đầu vào. Điều tương tự cũng đúng với `X_valid`, và cả với `X_test` và `X_new` khi bạn gọi `evaluate()` hoặc `predict()`:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)
model.compile(loss="mse", optimizer=optimizer, metrics=
    ["RootMeanSquaredError"])

X_train_wide, X_train_deep = X_train[:, :5], X_train[:, 2:]
X_valid_wide, X_valid_deep = X_valid[:, :5], X_valid[:, 2:]
X_test_wide, X_test_deep = X_test[:, :5], X_test[:, 2:]
X_new_wide, X_new_deep = X_test_wide[:3], X_test_deep[:3]

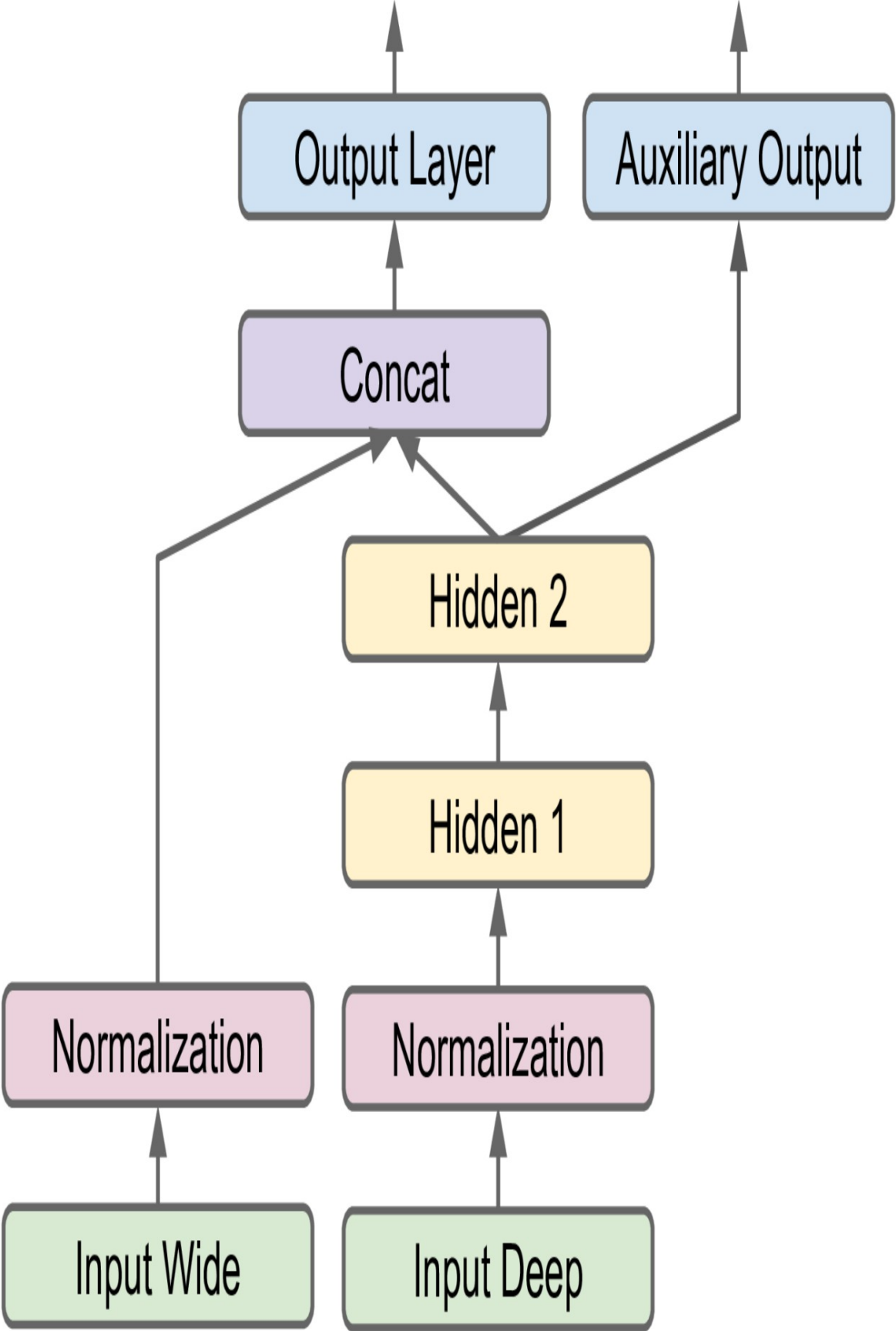
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep)
history = model.fit((X_train_wide, X_train_deep), y_train,
    epochs=20,
                    validation_data=((X_valid_wide,
mse_test =
model.evaluate((X_test_wide, X_valid_deep), y_valid)) mse_test =
model.evaluate((X_test_wide, X_test_deep), y_test) y_pred = model.predict((X_new_wide, X_new_deep))
```

MẸO

Thay vì truyền một tuple (`X_train_wide`, `X_train_deep`), bạn có thể truyền một dictionary `{"input_wide": X_train_wide, "input_deep": X_train_deep}`, với điều kiện bạn đã thiết lập `name="input_wide"` và `name="input_deep"` khi tạo các input. Điều này rất được khuyến khích khi có nhiều input, để làm rõ mã và tránh sai thứ tự.

Ngoài ra, còn có nhiều trường hợp sử dụng mà bạn có thể muốn có nhiều đầu ra:

- Nhiệm vụ có thể yêu cầu điều đó. Ví dụ, bạn có thể muốn xác định vị trí và phân loại đối tượng chính trong một bức ảnh. Đây vừa là nhiệm vụ hồi quy vừa là nhiệm vụ phân loại.
- Tương tự, bạn có thể có nhiều nhiệm vụ độc lập dựa trên cùng một dữ liệu. Chắc chắn, bạn có thể huấn luyện một mạng nơ-ron cho mỗi nhiệm vụ, nhưng trong nhiều trường hợp, bạn sẽ nhận được kết quả tốt hơn trên tất cả các nhiệm vụ bằng cách huấn luyện một mạng nơ-ron duy nhất với một đầu ra cho mỗi nhiệm vụ. Điều này là do mạng nơ-ron có thể học được các đặc điểm trong dữ liệu hữu ích cho nhiều nhiệm vụ. Ví dụ, bạn có thể thực hiện phân loại đa nhiệm trên ảnh khuôn mặt, sử dụng một đầu ra để phân loại biểu cảm khuôn mặt của người đó (cười, ngạc nhiên, v.v.) và một đầu ra khác để xác định xem họ có đeo kính hay không.
- Một trường hợp sử dụng khác là như một kỹ thuật điều chỉnh (tức là một ràng buộc huấn luyện với mục tiêu giảm thiểu hiện tượng quá khớp và do đó cải thiện khả năng khái quát hóa của mô hình). Ví dụ, bạn có thể muốn thêm một đầu ra phụ trợ trong kiến trúc mạng nơ-ron (xem [Hình 10-15](#)) để đảm bảo rằng phần cơ bản của mạng tự học được điều gì đó hữu ích, mà không cần dựa vào phần còn lại của mạng.



Hình 10-15. Xử lý nhiều đầu ra, trong ví dụ này là thêm một đầu ra phụ trợ để điều chỉnh.

Việc thêm một đầu ra bổ sung khá dễ dàng: chỉ cần kết nối nó với lớp thích hợp và thêm nó vào danh sách đầu ra của mô hình. Ví dụ, đoạn mã sau xây dựng mạng được thể hiện trong **Hình 10-15**:

```
[...] # Tương tự như trên, cho đến lớp đầu ra chính output =
tf.keras.layers.Dense(1)(concat) aux_output =
tf.keras.layers.Dense(1)(hidden2) model =
tf.keras.Model(inputs=[input_wide, input_deep],
                outputs=[output, aux_output])
```

Mỗi đầu ra sẽ cần một hàm mất mát riêng. Do đó, khi biên dịch mô hình, chúng ta nên truyền vào một danh sách các hàm mất mát. Nếu chúng ta truyền vào một hàm mất mát duy nhất, Keras sẽ giả định rằng cùng một hàm mất mát phải được sử dụng cho tất cả các đầu ra. Theo mặc định, Keras sẽ tính toán tất cả các hàm mất mát và chỉ đơn giản là cộng chúng lại để có được hàm mất mát cuối cùng được sử dụng để huấn luyện. Vì chúng ta quan tâm nhiều hơn đến đầu ra chính so với đầu ra phụ (vì nó chỉ được sử dụng để điều chỉnh), chúng ta muốn gán trọng số lớn hơn nhiều cho hàm mất mát của đầu ra chính. May mắn thay, có thể thiết lập tất cả các trọng số hàm mất mát khi biên dịch mô hình:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3) model.compile(loss=["mse",
"mse"), loss_weights=(0.9, 0.1), optimizer=optimizer, metrics=["RootMeanSquaredError"])
```

MẸO

Thay vì truyền một tuple `loss=("mse", "mse")`, bạn có thể truyền một dictionary `loss={"output": "mse", "aux_output": "mse"}`, giả sử bạn đã tạo các lớp đầu ra với `name="output"` và `name="aux_output"`. Giống như đối với đầu vào, điều này làm rõ mã và tránh lỗi khi có nhiều đầu ra. Bạn cũng có thể truyền một dictionary cho `loss_weights`.

Khi huấn luyện mô hình, bạn cần cung cấp nhãn cho mỗi đầu ra. Trong ví dụ này, đầu ra chính và đầu ra phụ nên cố gắng

Chúng dự đoán cùng một kết quả, vì vậy chúng nên sử dụng cùng một nhãn. Vì vậy, thay vì truyền `y_train`, bạn cần truyền `(y_train, y_train)`, hoặc một từ điển `{"output": y_train, "aux_output": y_train}` nếu các đầu ra được đặt tên là `"output"` và `"aux_output"`. Điều tương tự cũng áp dụng cho `y_valid` và `y_test`:

```
norm_layer_wide.adapt(X_train_wide)
norm_layer_deep.adapt(X_train_deep) history =
model.fit( (X_train_wide,
           X_train_deep), (y_train, y_train), epochs=20, validation_data=((X_valid_wide,
                                                                           X_valid_deep), (y_valid, y_valid)) )
```

Khi đánh giá mô hình, Keras trả về tổng có trọng số của các tổn thất, cũng như tất cả các tổn thất và chỉ số riêng lẻ:

```
eval_results = model.evaluate((X_test_wide, X_test_deep), (y_test, y_test))
weighted_sum_of_losses,
main_loss, aux_loss, main_rmse, aux_rmse = eval_results
```

MẸO

Nếu bạn đặt `return_dict=True`, thì `evaluate()` sẽ trả về một từ điển thay vì một tuple lớn.

Tương tự, phương thức `predict()` sẽ trả về các dự đoán cho mỗi đầu ra:

```
y_pred_main, y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Phương thức `predict()` trả về một tuple, và nó không có đối số `return_dict` để thay thế bằng một dictionary, nhưng bạn có thể tạo một dictionary bằng cách sử dụng `model.output_names`:

```
y_pred_tuple = model.predict((X_new_wide, X_new_deep)) y_pred =
dict(zip(model.output_names, y_pred_tuple))
```


Như bạn thấy, bạn có thể xây dựng đủ loại kiến trúc với Functional API. Hãy cùng xem một cách cuối cùng để xây dựng các mô hình Keras.

Sử dụng API kế thừa lớp con để xây dựng các mô hình động

Cả API tuần tự và API chức năng đều mang tính khai báo: bạn bắt đầu bằng cách khai báo các lớp bạn muốn sử dụng và cách chúng nên được kết nối, và chỉ sau đó bạn mới có thể bắt đầu cung cấp dữ liệu cho mô hình để huấn luyện hoặc suy luận. Điều này có nhiều ưu điểm: mô hình có thể dễ dàng được lưu, sao chép và chia sẻ; cấu trúc của nó có thể được hiển thị và phân tích; khung có thể suy luận hình dạng và kiểm tra kiểu dữ liệu, do đó lỗi có thể được phát hiện sớm (tức là trước khi bất kỳ dữ liệu nào được đưa vào mô hình). Việc gỡ lỗi cũng khá đơn giản, vì toàn bộ mô hình là một đồ thị tĩnh của các lớp.

Nhưng mặt trái của vấn đề là: nó tĩnh. Một số mô hình liên quan đến vòng lặp, hình dạng thay đổi, phân nhánh có điều kiện và các hành vi động khác. Trong những trường hợp như vậy, hoặc đơn giản là nếu bạn thích phong cách lập trình mệnh lệnh hơn, thì API kế thừa (Subclassing API) là dành cho bạn.

Bạn phải kế thừa lớp `Model`, tạo các lớp cần thiết trong hàm tạo và sử dụng chúng để thực hiện các phép tính mong muốn trong phương thức `call()`. Ví dụ, việc tạo một thể hiện của lớp `WideAndDeepModel` sau đây sẽ cho chúng ta một mô hình tương đương với mô hình mà chúng ta vừa xây dựng bằng Functional API:

```
class WideAndDeepModel(tf.keras.Model):
    def __init__(self, units=30,
                 activation="relu", **kwargs):
        super().__init__(**kwargs) # cần thiết để hỗ trợ đặt tên cho mô hình

        self.norm_layer_wide = tf.keras.layers.Normalization()
        self.norm_layer_deep = tf.keras.layers.Normalization()
        self.hidden1 = tf.keras.layers.Dense(units,
        kích hoạt = kích hoạt)
        self.hidden2 = tf.keras.layers.Dense(units,
        kích hoạt = kích hoạt)
        self.main_output = tf.keras.layers.Dense(1)
        self.aux_output = tf.keras.layers.Dense(1)

    def call(self, inputs):
        input_wide, input_deep = inputs
```

```

        norm_wide = self.norm_layer_wide(input_wide)
        norm_deep = self.norm_layer_deep(input_deep)
        hidden1 = self.hidden1(norm_deep)
        hidden2 = self.hidden2(hidden1)
        concat = tf.keras.layers.concatenate([norm_wide,
hidden2])
        output = self.main_output(concat)
        aux_output = self.aux_output(hidden2)
        return output, aux_output

model = WideAndDeepModel(30, activation="relu",
name="my_cool_model")

```

Ví dụ này trông rất giống với Functional API, ngoại trừ việc chúng ta tách biệt việc tạo các lớp trong hàm tạo¹⁷ khỏi việc sử dụng chúng trong phương thức call(). Chúng ta cũng không cần tạo các đối tượng Input: chúng ta chỉ sử dụng đối số input cho phương thức call().

Giờ đây, khi bạn đã có một thể hiện của mô hình, bạn có thể biên dịch nó, điều chỉnh các lớp chuẩn hóa của nó (ví dụ: sử dụng model.norm_layer_wide.adapt(...) và model.norm_layer_deep.adapt(...)), huấn luyện nó, đánh giá nó và sử dụng nó để đưa ra dự đoán, giống hệt như cách bạn đã làm với Functional API.

Điểm khác biệt lớn của API này là bạn có thể làm hầu hết mọi thứ mình muốn trong phương thức call(): vòng lặp for, câu lệnh if, các phép toán TensorFlow cấp thấp—giới hạn chỉ là trí tưởng tượng của bạn (xem [Chương 12](#))! Điều này làm cho nó trở thành một API tuyệt vời khi thử nghiệm các ý tưởng mới, đặc biệt là đối với các nhà nghiên cứu. Tuy nhiên, tính linh hoạt bổ sung này cũng đi kèm với một cái giá: kiến trúc của mô hình được ẩn bên trong phương thức call(), vì vậy Keras không thể dễ dàng kiểm tra nó, mô hình không thể được sao chép bằng tf.keras.models.clone_model(), và khi bạn gọi phương thức summary(), bạn chỉ nhận được một danh sách các lớp, mà không có bất kỳ thông tin nào về cách chúng được kết nối với nhau. Hơn nữa, Keras không thể kiểm tra các kiểu và hình dạng trước đó, và dễ mắc lỗi hơn. Vì vậy, trừ khi bạn thực sự cần tính linh hoạt bổ sung đó, bạn nên sử dụng API Sequential hoặc API Functional.

MẸO

Các mô hình Keras có thể được sử dụng giống như các lớp thông thường, vì vậy bạn có thể dễ dàng kết hợp chúng để xây dựng các kiến trúc phức tạp.

Giờ bạn đã biết cách xây dựng và huấn luyện mạng nơ-ron bằng Keras, chắc chắn bạn sẽ muốn lưu chúng lại!

Lưu trữ và khôi phục mô hình

Việc lưu một mô hình Keras đã được huấn luyện rất đơn giản:

```
model.save("my_keras_model", save_format="tf")
```

Khi bạn thiết lập `save_format="tf"`, Keras sẽ lưu mô hình bằng định dạng SavedModel của TensorFlow : đây là một thư mục (với tên đã cho) chứa một số tệp và thư mục con. Cụ thể, tệp `saved_model.pb` chứa kiến trúc và logic của mô hình dưới dạng đồ thị tính toán được tuần tự hóa, vì vậy bạn không cần triển khai mã nguồn của mô hình để sử dụng nó trong môi trường sản xuất, chỉ cần SavedModel là đủ (chúng ta sẽ xem cách hoạt động này trong [Chương 12](#)). Tệp `keras_metadata.pb` chứa thông tin bổ sung cần thiết cho Keras.

Thư mục con variables chứa tất cả các giá trị tham số (bao gồm trọng số kết nối, độ lệch, thống kê chuẩn hóa và các tham số của trình tối ưu hóa), có thể được chia thành nhiều tệp nếu mô hình rất lớn. Cuối cùng, thư mục assets có thể chứa các tệp bổ sung, ví dụ như các mẫu dữ liệu, tên đặc trưng, tên lớp, v.v. Theo mặc định, thư mục assets trống. Vì trình tối ưu hóa cũng được lưu, bao gồm cả các siêu tham số và bất kỳ trạng thái nào của nó, sau khi tải mô hình, bạn có thể tiếp tục huấn luyện nếu muốn.

GHI CHÚ

Nếu bạn đặt `save_format="h5"` hoặc sử dụng tên tệp kết thúc bằng `.h5`, `.hdf5` hoặc `.keras`, thì Keras sẽ lưu mô hình vào một tệp duy nhất bằng định dạng dành riêng cho Keras dựa trên định dạng HDF5. Tuy nhiên, hầu hết các công cụ triển khai TensorFlow yêu cầu định dạng `SaveDeleteModel`.

Thông thường, bạn sẽ có một tập lệnh để huấn luyện mô hình và lưu lại, cùng với một hoặc nhiều tập lệnh (hoặc dịch vụ web) để tải mô hình và sử dụng nó để đánh giá hoặc đưa ra dự đoán. Việc tải mô hình cũng dễ dàng như việc lưu mô hình:

```
model = tf.keras.models.load_model("my_keras_model") y_pred_main,
y_pred_aux = model.predict((X_new_wide, X_new_deep))
```

Bạn cũng có thể sử dụng `save_weights()` và `load_weights()` để lưu và tải riêng các giá trị tham số. Điều này bao gồm trọng số kết nối, độ lệch, số liệu thống kê tiền xử lý, trạng thái tối ưu hóa, v.v. Các giá trị tham số được lưu trong một hoặc nhiều tệp như `my_weights.data-00004-of-00052`, cộng thêm một tệp chỉ mục như `my_weights.index`.

Việc chỉ lưu trọng số sẽ nhanh hơn và sử dụng ít dung lượng ổ đĩa hơn so với việc lưu toàn bộ mô hình, vì vậy nó rất lý tưởng để lưu các điểm kiểm tra nhanh trong quá trình huấn luyện. Nếu bạn đang huấn luyện một mô hình lớn và quá trình này mất hàng giờ hoặc nhiều ngày, thì bạn phải lưu các điểm kiểm tra thường xuyên trong trường hợp máy tính bị lỗi. Nhưng làm thế nào bạn có thể yêu cầu phương thức `fit()` lưu các điểm kiểm tra? Hãy sử dụng các hàm gọi lại (callback).

Sử dụng hàm gọi lại

Phương thức `fit()` chấp nhận một đối số `callbacks` cho phép bạn chỉ định một danh sách các đối tượng mà Keras sẽ gọi trước và sau khi huấn luyện, trước và sau mỗi epoch, và thậm chí trước và sau khi xử lý mỗi batch. Ví dụ, hàm gọi lại `ModelCheckpoint` lưu các checkpoint của mô hình theo định kỳ trong quá trình huấn luyện, theo mặc định là vào cuối mỗi epoch:

```
checkpoint_cb =
tf.keras.callbacks.ModelCheckpoint("my_checkpoints",

save_weights_only=True)
history = model.fit(..., callbacks=[checkpoint_cb])
```

Hơn nữa, nếu bạn sử dụng tập dữ liệu kiểm định trong quá trình huấn luyện, bạn có thể đặt `save_best_only=True` khi tạo `ModelCheckpoint`. Trong trường hợp này, mô hình của bạn sẽ chỉ được lưu khi hiệu suất của nó trên tập dữ liệu kiểm định là tốt nhất cho đến nay. Bằng cách này, bạn không cần phải lo lắng về việc huấn luyện quá lâu và hiện tượng quá khớp trên tập dữ liệu huấn luyện: chỉ cần khôi phục mô hình đã lưu cuối cùng sau khi huấn luyện, và đây sẽ là mô hình tốt nhất trên tập dữ liệu kiểm định. Đây là một cách để thực hiện dừng sớm (được giới thiệu trong [Chương 4](#)), nhưng nó sẽ không thực sự dừng quá trình huấn luyện.

Một cách khác là sử dụng hàm gọi lại `EarlyStopping`. Hàm này sẽ tạm dừng quá trình huấn luyện khi không thấy có tiến triển nào trên tập dữ liệu kiểm chứng trong một số epoch nhất định (được xác định bởi tham số `patience`), và nếu bạn đặt `restore_best_weights=True`, nó sẽ quay lại mô hình tốt nhất ở cuối quá trình huấn luyện. Bạn có thể kết hợp cả hai hàm gọi lại để lưu các điểm kiểm tra của mô hình trong trường hợp máy tính bị lỗi, và tạm dừng quá trình huấn luyện sớm khi không còn tiến triển nào nữa, để tránh lãng phí thời gian và tài nguyên cũng như giảm thiểu hiện tượng quá khớp (overfitting):

```
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=10,

restore_best_weights=True)
history = model.fit(..., callbacks=[checkpoint_cb,
early_stopping_cb])
```

Số lượng epoch có thể được đặt thành một giá trị lớn vì quá trình huấn luyện sẽ tự động dừng lại khi không còn tiến triển nào nữa (chỉ cần đảm bảo tốc độ học không quá nhỏ, nếu không nó có thể tiếp tục tiến triển chậm cho đến khi kết thúc). Hàm `EarlyStopping` sẽ lưu trữ trọng số của mô hình tốt nhất trong RAM và sẽ khôi phục chúng cho bạn khi quá trình huấn luyện kết thúc.

MẸO

Còn rất nhiều hàm gọi lại khác có sẵn trong gói `tf.keras.callbacks`.

Nếu bạn cần quyền kiểm soát cao hơn, bạn có thể dễ dàng tự viết các hàm gọi lại tùy chỉnh của riêng mình. Ví dụ, hàm gọi lại tùy chỉnh sau đây sẽ hiển thị tỷ lệ giữa tổn thất xác thực và tổn thất huấn luyện trong quá trình huấn luyện (ví dụ, để phát hiện hiện tượng quá khớp):

```
class PrintValTrainRatioCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs):
        tỷ lệ = log["val_loss"] / log["loss"]
        print(f"Epoch={epoch}, val/train={ratio:.2f}")
```

Như bạn có thể đoán được, bạn có thể triển khai các phương thức `on_train_begin()`, `on_train_end()`, `on_epoch_begin()`, `on_epoch_end()`, `on_batch_begin()` và `on_batch_end()`. Các hàm callback cũng có thể được sử dụng trong quá trình đánh giá và dự đoán, nếu bạn cần chúng (ví dụ: để gỡ lỗi). Đối với quá trình đánh giá, bạn nên triển khai các phương thức `on_test_begin()`, `on_test_end()`, `on_test_batch_begin()` hoặc `on_test_batch_end()`, được gọi bởi phương thức `evaluate()`. Đối với quá trình dự đoán, bạn nên triển khai các phương thức `on_predict_begin()`, `on_predict_end()`, `on_predict_batch_begin()` hoặc `on_predict_batch_end()`, được gọi bởi phương thức `predict()`.

Giờ chúng ta hãy cùng xem xét thêm một công cụ nữa mà bạn chắc chắn nên có trong bộ công cụ của mình khi sử dụng Keras: TensorBoard.

Sử dụng TensorBoard để trực quan hóa

TensorBoard là một công cụ trực quan tương tác tuyệt vời mà bạn có thể sử dụng để xem các đường cong học tập trong quá trình huấn luyện, so sánh các đường cong và số liệu giữa nhiều lần chạy, trực quan hóa đồ thị tính toán, phân tích số liệu thống kê huấn luyện, xem hình ảnh do mô hình của bạn tạo ra, trực quan hóa dữ liệu đa chiều phức tạp được chiếu xuống không gian 3D và tự động phân cụm.

Bạn có thể lập hồ sơ mạng của mình (tức là đo tốc độ để xác định các điểm nghẽn), và hơn thế nữa!

TensorBoard được cài đặt tự động khi bạn cài đặt TensorFlow.

Tuy nhiên, bạn sẽ cần một plugin TensorBoard để trực quan hóa dữ liệu phân tích hiệu năng. Nếu bạn đã làm theo hướng dẫn cài đặt tại <https://homl.info/install> để chạy mọi thứ cục bộ, bạn đã cài đặt sẵn plugin, nhưng nếu bạn đang sử dụng Colab, thì bạn phải chạy lệnh sau:

```
%pip install -q -U tensorboard-plugin-profile
```

Để sử dụng TensorBoard, bạn phải sửa đổi chương trình của mình sao cho nó xuất dữ liệu bạn muốn trực quan hóa ra các tệp nhật ký nhị phân đặc biệt được gọi là tệp sự kiện. Mỗi bản ghi dữ liệu nhị phân được gọi là bản tóm tắt. Máy chủ TensorBoard sẽ giám sát thư mục nhật ký và tự động nhận biết các thay đổi và cập nhật hình ảnh trực quan: điều này cho phép bạn trực quan hóa dữ liệu trực tiếp (với độ trễ ngắn), chẳng hạn như đường cong học tập trong quá trình huấn luyện. Nói chung, bạn nên trở máy chủ TensorBoard đến thư mục nhật ký gốc và cấu hình chương trình của mình sao cho nó ghi vào một thư mục con khác nhau mỗi khi chạy. Bằng cách này, cùng một phiên bản máy chủ TensorBoard sẽ cho phép bạn trực quan hóa và so sánh dữ liệu từ nhiều lần chạy chương trình của bạn mà không làm lẫn lộn mọi thứ.

Chúng ta hãy đặt tên thư mục nhật ký gốc là `my_logs`, và hãy định nghĩa một hàm nhỏ để tạo đường dẫn của thư mục con nhật ký dựa trên ngày giờ hiện tại, sao cho đường dẫn này khác nhau mỗi khi chạy:

```
from pathlib import Path
from time import strftime

def get_run_logdir(root_logdir="my_logs"):
    return Path(root_logdir) / strftime("run_%Y_%m_%d_%H_%M_%S")

run_logdir = get_run_logdir() # ví dụ:
my_logs/run_2022_08_01_17_25_59
```

Tin tốt là Keras cung cấp một hàm gọi lại `TensorBoard()` tiện lợi, hàm này sẽ tự động tạo thư mục nhật ký cho bạn (cùng với...).

(bao gồm cả các thư mục cha nếu cần), và nó sẽ tạo các tệp sự kiện và ghi lại.

Tóm tắt nội dung cho chúng trong quá trình huấn luyện. Điều này sẽ giúp đánh giá quá trình huấn luyện của mô hình.

và tổn thất xác thực và các chỉ số (trong trường hợp này là MSE và RMSE), và nó

Nó cũng sẽ lập hồ sơ mạng thần kinh của bạn. Cách sử dụng rất đơn giản:

```
tensorboard_cb = tf.keras.callbacks.TensorBoard(run_logdir,
                                                profile_batch=
(100, 200))
lịch sử = mô hình.fit(..., hàm gọi lại=[tensorboard_cb])
```

Và thế là xong! Trong ví dụ này, nó sẽ lập hồ sơ mạng.

Giữa các lô 100 và 200 trong kỷ nguyên đầu tiên. Tại sao lại là 100 và 200?

Thường thì mạng nơ-ron cần một vài lần xử lý để "khởi động", vì vậy

Bạn không nên lập hồ sơ quá sớm. Và việc lập hồ sơ cũng tốn khá nhiều tài nguyên, vì vậy...

Tốt nhất là không nên làm như vậy với từng mẻ sản phẩm.

Tiếp theo, hãy thử thay đổi tốc độ học từ 0.001 thành 0.002 và chạy đoạn mã.

Một lần nữa, với một thư mục con nhật ký mới. Cuối cùng bạn sẽ có một thư mục.

Cấu trúc tương tự như thế này:

```
nhật ký của tôi
├─ run_2022_08_01_17_25_59
│   └─ tàu hỏa
│       └─
│           events.out.tfevents.1659331561.my_host_name.42042.0.v2
│               └─ events.out.tfevents.1659331562.my_host_name.profile-
trống         └─ plugin
│               └─ hồ sơ
│                   └─ 2022_08_01_17_26_02
│                       └─ my_host_name.input_pipeline.pb
xác thực         └─ [...]
└─
    └─ events.out.tfevents.1659331562.my_host_name.42042.1.v2
        └─ run_2022_08_01_17_31_12
            └─ [...]
```

Mỗi lần chạy sẽ có một thư mục, và mỗi thư mục này lại chứa một thư mục con dành cho quá trình huấn luyện.

Hai thư mục, một dành cho nhật ký và một dành cho nhật ký xác thực. Cả hai đều chứa các tệp sự kiện và quá trình huấn luyện.

Các tệp nhật ký cũng bao gồm dấu vết phân tích hiệu năng.

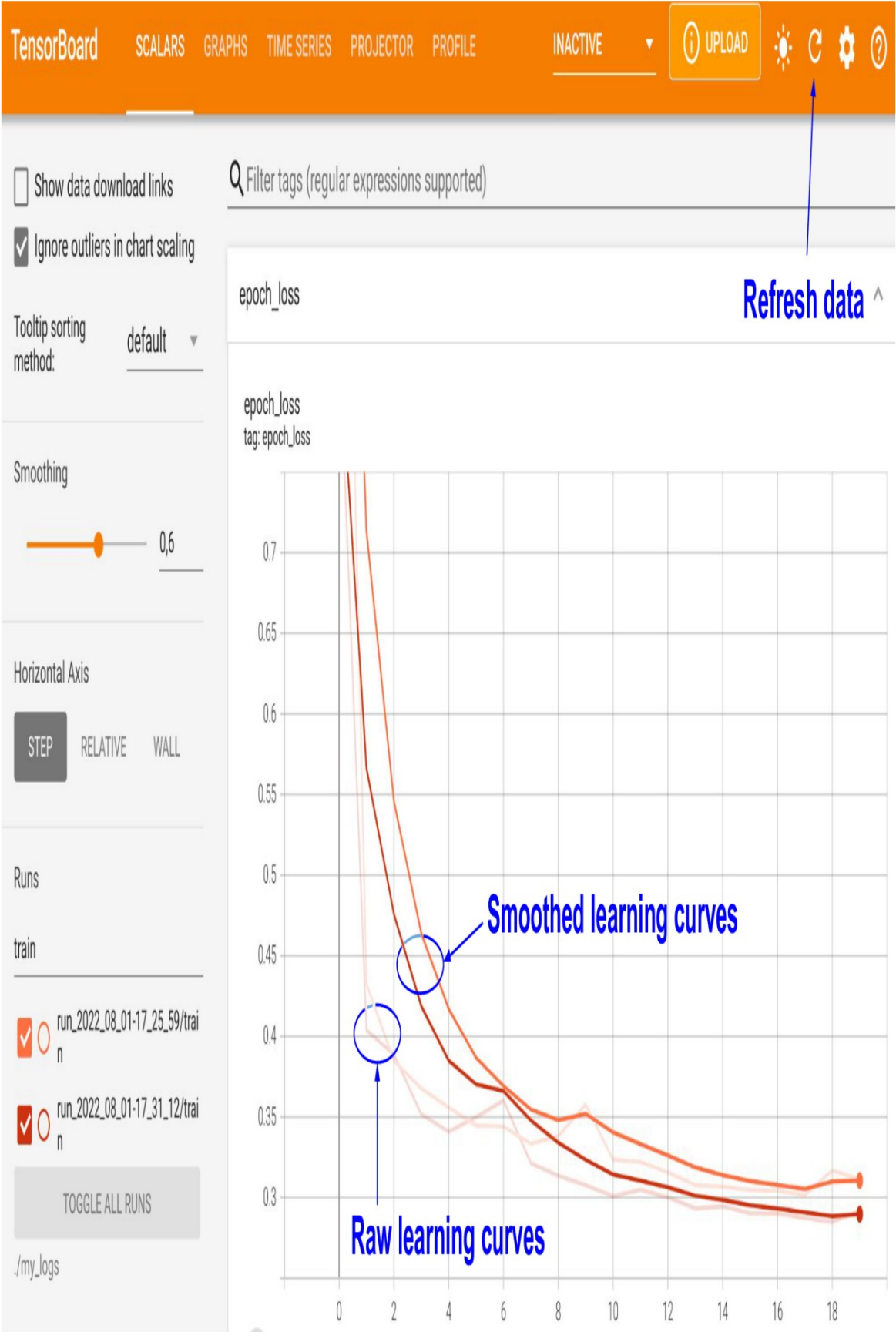
Giờ bạn đã chuẩn bị xong các tệp sự kiện, đã đến lúc khởi động máy chủ TensorBoard. Việc này có thể được thực hiện trực tiếp trong Jupyter hoặc Colab bằng cách sử dụng tiện ích mở rộng Jupyter cho TensorBoard, được cài đặt cùng với thư viện TensorBoard. Tiện ích mở rộng này đã được cài đặt sẵn trong Colab. Đoạn mã sau tải tiện ích mở rộng Jupyter cho TensorBoard, và dòng thứ hai khởi động máy chủ TensorBoard cho thư mục my_logs, kết nối với máy chủ này và hiển thị giao diện người dùng trực tiếp bên trong Jupyter. Máy chủ lắng nghe trên cổng TCP khả dụng đầu tiên lớn hơn hoặc bằng 6006 (hoặc bạn có thể đặt cổng mong muốn bằng tùy chọn --port).

```
%load_ext tensorboard
%tensorboard --logdir=./my_logs
```

MẸO

Nếu bạn đang chạy mọi thứ trên máy tính của mình, bạn cũng có thể khởi động TensorBoard bằng cách thực hiện lệnh `tensorboard --logdir=./my_logs` trong cửa sổ terminal. Trước tiên, bạn cần kích hoạt môi trường conda mà bạn đã sử dụng để cài đặt TensorBoard, và chuyển đến thư mục `handson-ml3`. Sau khi máy chủ khởi động, hãy truy cập <http://localhost:6006/>.

Giờ bạn sẽ thấy giao diện người dùng của TensorBoard. Nhấp vào tab SCALARS để xem các đường cong học tập (xem **Hình 10-16**). Ở phía dưới bên trái, chọn nhật ký bạn muốn trực quan hóa (ví dụ: nhật ký huấn luyện từ lần chạy đầu tiên và thứ hai), và nhấp vào giá trị `epoch_loss`. Lưu ý rằng tổn thất huấn luyện giảm khá tốt trong cả hai lần chạy, nhưng lần chạy thứ hai giảm nhanh hơn một chút nhờ tốc độ học tập cao hơn.



Hình 10-16. Trực quan hóa đường cong học tập với TensorBoard

Bạn cũng có thể hình dung toàn bộ đồ thị tính toán trong tab ĐỒ THỊ, các trọng số đã học được chiếu lên không gian 3D trong tab MÁY CHIẾU và các dấu vết phân tích hiệu năng trong tab HỒ SƠ. Hàm gọi lại TensorBoard() cũng có các tùy chọn để ghi nhật ký dữ liệu bổ sung (xem tài liệu để biết thêm chi tiết). Bạn có thể nhấp vào nút làm mới () ở góc trên bên phải để TensorBoard làm mới dữ liệu, và bạn có thể nhấp vào nút cài đặt () để kích hoạt tự động làm mới và chỉ định khoảng thời gian làm mới.

Ngoài ra, TensorFlow cung cấp API cấp thấp hơn trong gói tf.summary. Đoạn mã sau tạo một SummaryWriter bằng cách sử dụng hàm create_file_writer(), và nó sử dụng trình ghi này như một ngữ cảnh Python để ghi nhật ký các giá trị vô hướng, biểu đồ, hình ảnh, âm thanh và văn bản, tất cả đều có thể được trực quan hóa bằng TensorBoard:

```
test_logdir = get_run_logdir() writer
= tf.summary.create_file_writer(str(test_logdir)) with
writer.as_default(): for step
    in range(1, 1000 + 1):
        tf.summary.scalar("my_scalar", np.sin(step / 10), step=step)

        dữ liệu = (np.random.randn(100) + 2) * bước / 100 # nhận được
        lớn hơn
        tf.summary.histogram("my_hist", data, buckets=50, step=step)

        hình ảnh = np.random.rand(2, 32, 32, 3) * bước / 1000 #
        trở nên sáng hơn
        tf.summary.image("my_images", images, step=step)

        texts = ["Bước là " + str(step) + str(step), "Bình phương của nó là " +
        ** 2)]
        tf.summary.text("my_text", texts, step=step)

        sóng sin = tf.math.sin(tf.range(12000) / 48000) * 2 *
        np.pi * bước)
        audio = tf.reshape(tf.cast(sine_wave, tf.float32), [1,
        -1, 1])
        tf.summary.audio("my_audio", audio, sample_rate=48000, step=step)
```

Nếu bạn chạy đoạn mã này và nhấn nút làm mới trong TensorBoard, bạn sẽ thấy một số tab xuất hiện: IMAGES, AUDIO, DISTRIBUTIONS, HISTOGRAMS và TEXT.

Hãy thử nhấn vào tab IMAGES và sử dụng thanh trượt phía trên mỗi hình ảnh để xem hình ảnh ở các mốc thời gian khác nhau.

Tương tự, hãy vào tab AUDIO và thử nghe các đoạn âm thanh ở các mốc thời gian khác nhau. Như bạn thấy, TensorBoard là một công cụ hữu ích ngay cả ngoài phạm vi TensorFlow hay Deep Learning.

MẸO

Bạn có thể chia sẻ kết quả của mình trực tuyến bằng cách đăng tải chúng lên <https://tensorboard.dev>. Để thực hiện việc này, chỉ cần chạy lệnh `!tensorboard dev upload --logdir ./my_logs`. Lần đầu tiên, hệ thống sẽ yêu cầu bạn chấp nhận các điều khoản và điều kiện cũng như xác thực. Sau đó, nhật ký của bạn sẽ được tải lên và bạn sẽ nhận được một liên kết cố định để xem kết quả trong giao diện TensorBoard.

Hãy cùng tóm tắt những gì bạn đã học được trong chương này: bạn đã biết mạng nơ-ron xuất phát từ đâu, MLP là gì và cách sử dụng nó cho phân loại và hồi quy, cách sử dụng API Sequential của Keras để xây dựng MLP, và cách sử dụng API Functional hoặc API Subclassing để xây dựng các kiến trúc mô hình phức tạp hơn, bao gồm cả các mô hình Wide & Deep, cũng như các mô hình có nhiều đầu vào và đầu ra. Bạn cũng đã học cách lưu và khôi phục mô hình và cách sử dụng các hàm gọi lại (callback) để tạo điểm kiểm tra, dừng sớm, và nhiều hơn nữa. Cuối cùng, bạn đã học cách sử dụng TensorBoard để trực quan hóa. Bạn đã có thể bắt đầu sử dụng mạng nơ-ron để giải quyết nhiều vấn đề! Tuy nhiên, bạn có thể tự hỏi làm thế nào để chọn số lớp ẩn, số nơ-ron trong mạng và tất cả các siêu tham số khác. Hãy cùng xem xét điều này ngay bây giờ.

Tinh chỉnh mạng nơ-ron Siêu tham số

Tính linh hoạt của mạng nơ-ron cũng chính là một trong những nhược điểm chính của chúng: có rất nhiều siêu tham số cần điều chỉnh. Bạn không chỉ có thể sử dụng bất kỳ kiến trúc mạng nào có thể tưởng tượng được, mà ngay cả trong một mạng đa lớp (MLP) cơ bản, bạn cũng có thể thay đổi số lớp, số nơ-ron và loại hàm kích hoạt được sử dụng trong mỗi lớp, logic khởi tạo trọng số, loại bộ tối ưu hóa được sử dụng, tốc độ học của nó, kích thước lô (batch size), và nhiều hơn nữa. Làm thế nào để bạn biết được sự kết hợp nào của các siêu tham số là tốt nhất cho nhiệm vụ của mình?

Một lựa chọn khác là chuyển đổi mô hình Keras của bạn thành trình ước lượng Scikit-Learn, sau đó sử dụng GridSearchCV hoặc RandomizedSearchCV để tinh chỉnh.

Các siêu tham số, như bạn đã làm trong [Chương 2](#). Để làm điều này, bạn có thể sử dụng các lớp bao bọc KerasRegressor và KerasClassifier từ thư viện SciKeras (xem <https://github.com/adriangb/scikeras>) (để biết thêm chi tiết). Tuy nhiên, có một cách tốt hơn: bạn có thể sử dụng thư viện Keras Tuner, một thư viện điều chỉnh siêu tham số cho các mô hình Keras. Nó cung cấp một số chiến lược điều chỉnh, có khả năng tùy chỉnh cao và tích hợp tuyệt vời với TensorBoard. Hãy cùng xem cách sử dụng nó.

Nếu bạn đã làm theo hướng dẫn cài đặt tại <https://hom1.info/install> để chạy mọi thứ cục bộ, bạn đã cài đặt Keras Tuner, nhưng nếu bạn đang sử dụng Colab, thì bạn phải chạy `%pip install -q -U keras-tuner`. Tiếp theo, nhập `keras_tuner`, thường là `kt`, sau đó viết một hàm xây dựng, biên dịch và trả về một mô hình Keras. Hàm phải nhận một đối tượng `kt.HyperParameters` làm đối số, mà nó có thể sử dụng để định nghĩa các siêu tham số (số nguyên, số thực, chuỗi, v.v.) cùng với phạm vi giá trị có thể có của chúng, và các siêu tham số này có thể được sử dụng để xây dựng và biên dịch mô hình. Ví dụ, hàm sau xây dựng và biên dịch một MLP để phân loại hình ảnh Fashion MNIST, sử dụng các siêu tham số như số lớp ẩn (`n_hidden`), số nơ-ron trên mỗi lớp (`n_neurons`), tốc độ học (`learning_rate`) và loại trình tối ưu hóa cần sử dụng (`optimizer`):

```

import keras_tuner as kt

def build_model(hp):
    n_hidden = hp.Int("n_hidden", min_value=0, max_value=8, default=2)
    n_neurons =
        hp.Int("n_neurons", min_value=16, max_value=256)
        learning_rate =
            hp.Float("learning_rate", min_value=1e-4, max_value=1e-2,

                                sampling="log")
        optimizer = hp.Choice("optimizer", values=["sgd", "adam"])
        if optimizer == "sgd":
            optimizer =
                tf.keras.optimizers.SGD(learning_rate=learning_rate)
        khác:
            optimizer =
                tf.keras.optimizers.Adam(learning_rate=learning_rate)

    model = tf.keras.Sequential()
    model.add(tf.keras.layers.Flatten())
    for
        _ trong phạm vi (n_hidden):
            model.add(tf.keras.layers.Dense(n_neurons,
activation="relu"))
    model.add(tf.keras.layers.Dense(10, activation="softmax"))
    model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer,

                                số liệu=["độ chính xác"])

    mô hình trả về

```

Phần đầu tiên của hàm định nghĩa các siêu tham số. Ví dụ, `hp.Int("n_hidden", min_value=0, max_value=8, default=2)` kiểm tra xem siêu tham số có tên "n_hidden" đã tồn tại trong đối tượng HyperParameters `hp` hay chưa, và nếu có thì trả về giá trị của nó. Nếu không, thì nó đăng ký một siêu tham số kiểu số nguyên mới có tên "n_hidden", với các giá trị có thể từ 0 đến 8 (bao gồm cả 0 và 8), và trả về giá trị mặc định, trong trường hợp này là 2 (khi giá trị mặc định không được đặt, thì `min_value` được trả về). Siêu tham số "n_neurons" được đăng ký theo cách tương tự. Siêu tham số "learning_rate" được đăng ký dưới dạng số thực có giá trị từ 10^{-4} đến 10^{-2} , lấy mẫu ("log"), tốc độ học ở tất cả các thang đo sẽ được lấy mẫu 10^{-4} , và kể từ đầu nhau.

Cuối cùng, siêu tham số của trình tối ưu hóa được đăng ký với hai giá trị có thể: "sgd" hoặc "adam" (giá trị mặc định là giá trị đầu tiên, tức là...)

(Trong trường hợp này là "sgd"). Tùy thuộc vào giá trị của optimizer, chúng ta sẽ tạo ra một optimizer SGD hoặc optimizer Adam với tốc độ học đã cho.

Phần thứ hai của hàm chỉ đơn giản là xây dựng mô hình bằng cách sử dụng các giá trị siêu tham số. Nó tạo ra một mô hình Sequential bắt đầu với một lớp Flatten, tiếp theo là số lượng lớp ẩn được yêu cầu (được xác định bởi siêu tham số n_hidden) sử dụng hàm kích hoạt ReLU, và một lớp đầu ra với 10 nơ-ron (một nơ-ron cho mỗi lớp) và sử dụng hàm kích hoạt softmax. Cuối cùng, hàm biên dịch mô hình và trả về nó.

Nếu bạn muốn thực hiện tìm kiếm ngẫu nhiên cơ bản, bạn có thể tạo một bộ điều chỉnh kt.RandomSearch, truyền hàm build_model vào hàm tạo và gọi phương thức search() của bộ điều chỉnh đó:

```
random_search_tuner = kt.RandomSearch(
    build_model, objective="val_accuracy", max_trials=5,
    overwrite=True,
    directory="my_fashion_mnist", project_name="my_rnd_search",
    seed=42)
random_search_tuner.search(X_train, y_train, epochs=10,
                           validation_data=(X_valid, y_valid))
```

Bộ điều chỉnh RandomSearch trước tiên gọi hàm build_model() một lần với đối tượng Hyperparameters rỗng, chỉ để thu thập tất cả các thông số siêu tham số. Sau đó, trong ví dụ này, nó chạy 5 lần thử nghiệm, và với mỗi lần thử nghiệm, nó xây dựng một mô hình sử dụng các siêu tham số được lấy mẫu ngẫu nhiên trong phạm vi tương ứng của chúng, sau đó nó huấn luyện mô hình đó trong 10 epoch và lưu nó vào một thư mục con của thư mục my_fashion_mnist/my_rnd_search.

Vì overwrite=True, thư mục my_rnd_search sẽ bị xóa trước khi quá trình huấn luyện bắt đầu. Nếu bạn chạy đoạn mã này lần thứ hai nhưng với overwrite=False và max_trials=10, trình điều chỉnh sẽ tiếp tục điều chỉnh từ nơi nó đã dừng lại, chạy thêm 5 lần thử nghiệm nữa: điều này có nghĩa là bạn không cần phải chạy tất cả các lần thử nghiệm cùng một lúc. Cuối cùng, vì objective được đặt là "val_accuracy", trình điều chỉnh sẽ ưu tiên các mô hình có độ chính xác thực cao hơn, vì vậy sau khi trình điều chỉnh hoàn tất quá trình tìm kiếm, bạn có thể nhận được các mô hình tốt nhất như sau:

```
top3_models = random_search_tuner.get_best_models(num_models=3) best_model = top3_models[0]
```

Bạn cũng có thể gọi `get_best_hyperparameters()` để lấy `kt.HyperParameters` của các mô hình tốt nhất:

```
>>> top3_params =
random_search_tuner.get_best_hyperparameters(num_trials=3) >>> top3_params[0].values
# giá trị siêu tham số tốt nhất {'n_hidden': 5,

'n_neurons': 70,
'learning_rate': 0.00041268008323824807, 'optimizer':
'adam'}
```

Mỗi bộ điều chỉnh được hướng dẫn bởi một cái gọi là "thần linh": trước mỗi lần thử, bộ điều chỉnh sẽ hỏi thần linh xem lần thử tiếp theo nên là gì. Tìm kiếm ngẫu nhiên

Tuner sử dụng `RandomSearchOracle`, khá đơn giản: nó chỉ chọn ngẫu nhiên lượt thử tiếp theo, như chúng ta đã thấy trước đó. Vì oracle theo dõi tất cả các lượt thử, bạn có thể yêu cầu nó cung cấp cho bạn lượt thử tốt nhất, và bạn có thể hiển thị tóm tắt của lượt thử đó:

```
>>> best_trial =
random_search_tuner.oracle.get_best_trials(num_trials=1)[0] >>> best_trial.summary()
```

```
Tóm tắt kết quả thử
nghiệm: Siêu tham số:
n_hidden: 5
n_neurons: 70
learning_rate: 0.00041268008323824807 optimizer: adam Điểm số:
0.8736000061035156
```

Điều này hiển thị các siêu tham số tối ưu nhất (như đã đề cập trước đó), cũng như độ chính xác khi kiểm định. Bạn cũng có thể truy cập trực tiếp vào tất cả các chỉ số:

```
>>> best_trial.metrics.get_last_value("val_accuracy") 0.8736000061035156
```

Nếu bạn hài lòng với hiệu suất của mô hình tốt nhất, bạn có thể tiếp tục huấn luyện nó thêm vài epoch trên toàn bộ tập dữ liệu huấn luyện (`X_train_full` và

`y_train_full`), sau đó đánh giá nó trên tập dữ liệu thử nghiệm và triển khai lên môi trường sản xuất (xem [Chương 19](#)):

```
best_model.fit(X_train_full, y_train_full, epochs=10)
test_loss, test_accuracy = best_model.evaluate(X_test, y_test)
```

Trong một số trường hợp, bạn có thể muốn tinh chỉnh các siêu tham số xử lý dữ liệu trước hoặc các đối số của `model.fit()`, chẳng hạn như kích thước batch. Để làm điều này, bạn phải sử dụng một kỹ thuật hơi khác: thay vì viết hàm `build_model()`, bạn phải tạo lớp con của lớp `kt.HyperModel` và định nghĩa hai phương thức, `build()` và `fit()`. Phương thức `build()` thực hiện chính xác điều tương tự như hàm `build_model()`, và phương thức `fit()` nhận một đối tượng `HyperParameters` và một mô hình đã biên dịch làm đối số, cũng như tất cả các đối số của `model.fit()`, và nó phải huấn luyện mô hình và trả về đối tượng `History`. Điều quan trọng là, phương thức `fit()` có thể sử dụng các siêu tham số để quyết định cách xử lý dữ liệu trước, điều chỉnh kích thước batch, v.v. Ví dụ, lớp sau xây dựng cùng một mô hình như trước, với cùng các siêu tham số, nhưng nó cũng sử dụng một siêu tham số boolean "normalize" để kiểm soát việc có chuẩn hóa dữ liệu huấn luyện trước khi huấn luyện mô hình hay không:

```
class MyClassificationHyperModel(kt.HyperModel):
    def build(self, hp):
        return build_model(hp)

    def fit(self, hp, model, X, y, **kwargs):
        if hp.Boolean("normalize"):
            norm_layer = tf.keras.layers.Normalization()
            X = norm_layer(X)
        return model.fit(X, y, **kwargs)
```

Sau đó, bạn có thể truyền một thể hiện của lớp này cho bộ điều chỉnh mà bạn chọn, thay vì truyền hàm `build_model`. Ví dụ, hãy xây dựng một bộ điều chỉnh `kt.Hyperband` dựa trên một thể hiện của `MyClassificationHyperModel`:

```
hyperband_tuner =
    kt.Hyperband( MyClassificationHyperModel(),
        objective="val_accuracy", seed=42, max_epochs=10,
        factor=3, hyperband_iterations=2, overwrite=True, directory="my_fashion_mnist",
        project_name="hyperband")
```

Bộ điều chỉnh này tương tự như lớp HalvingRandomSearchCV mà chúng ta đã thảo luận trong **Chương 2**: nó bắt đầu bằng cách huấn luyện nhiều mô hình khác nhau trong vài epoch, sau đó loại bỏ các mô hình tệ nhất và chỉ giữ lại $1/\text{factor}$ mô hình hàng đầu (tức là một phần ba hàng đầu trong trường hợp này), lặp lại quá trình lựa chọn này cho đến khi chỉ còn lại một mô hình duy nhất. Tham số `max_epochs` kiểm soát số lượng epoch tối đa mà mô hình tốt nhất sẽ được huấn luyện. Toàn bộ quá trình được lặp lại hai lần trong trường hợp này (`hyperband_iterations=2`). Tổng số epoch huấn luyện trên tất cả các mô hình cho mỗi lần lặp hyperband là khoảng $\text{max_epochs} * (\log(\text{max_epochs}) / \log(\text{factor})) ** \text{epochs}$ trong ví dụ này. Các tham số khác giống như đối với `kt.RandomSearch`.

Bây giờ chúng ta hãy chạy bộ điều chỉnh Hyperband. Chúng ta sẽ sử dụng hàm gọi lại TensorBoard, lần này trở đến thư mục nhật ký gốc (bộ điều chỉnh sẽ tự động sử dụng một thư mục con khác nhau cho mỗi lần thử), cũng như hàm gọi lại EarlyStopping:

```
root_logdir = Path(hyperband_tuner.project_dir) / "tensorboard"
tensorboard_cb = tf.keras.callbacks.TensorBoard(root_logdir)
early_stopping_cb = tf.keras.callbacks.EarlyStopping(patience=2)
hyperband_tuner.search(X_train, y_train, epochs=10,
                       validation_data=(X_valid, y_valid),
                       callbacks=[early_stopping_cb,
                                   tensorboard_cb])
```

Bây giờ, nếu bạn mở TensorBoard, trở tham số `--logdir` đến thư mục `my_fashion_mnist/hyperband/tensorboard`, bạn sẽ thấy tất cả các kết quả thử nghiệm khi chúng được hiển thị. Hãy nhớ truy cập vào tab HPARAMS: tab này chứa tóm tắt tất cả các tổ hợp siêu tham số đã được thử, cùng với các chỉ số tương ứng. Lưu ý rằng có ba tab.

Bên trong tab HPARAMS: có chế độ xem dạng bảng, chế độ xem tọa độ song song và chế độ xem ma trận biểu đồ phân tán. Ở phần dưới của bảng điều khiển bên trái, bỏ chọn tất cả các chỉ số ngoại trừ `validation.epoch_accuracy`: điều này sẽ làm cho các biểu đồ rõ ràng hơn. Trong chế độ xem tọa độ song song, hãy thử chọn một phạm vi giá trị cao trên cột `validation.epoch_accuracy`: điều này sẽ chỉ lọc ra các tổ hợp siêu tham số đạt được hiệu suất tốt. Hãy nhấp chuột vào một trong các tổ hợp siêu tham số: các đường cong học tập tương ứng sẽ xuất hiện ở cuối trang. Dành chút thời gian để xem xét từng tab: chúng sẽ giúp bạn hiểu được tác động của từng siêu tham số đến hiệu năng, cũng như sự tương tác giữa các siêu tham số.

Hyperband thông minh hơn thuật toán tìm kiếm ngẫu nhiên thuần túy ở cách phân bổ tài nguyên, nhưng về bản chất, nó vẫn khám phá không gian siêu tham số một cách ngẫu nhiên: nhanh nhưng thô. Tuy nhiên, Keras Tuner cũng bao gồm một bộ điều chỉnh `kt.BayesianOptimization`: thuật toán này dần dần học được những vùng nào trong không gian siêu tham số hứa hẹn nhất bằng cách khớp một mô hình xác suất được gọi là quy trình Gaussian. Điều này cho phép nó dần dần tập trung vào các siêu tham số tốt nhất. Nhược điểm là thuật toán này có các siêu tham số riêng: α đại diện cho mức độ nhiễu mà bạn mong đợi trong các phép đo hiệu suất trên các lần thử (mặc định là 10^{-4}), và β chỉ định mức độ bạn muốn thuật toán khám phá, thay vì chỉ đơn giản là khai thác các vùng tốt đã biết trong không gian siêu tham số (mặc định là 2.6). Ngoài ra, bộ điều chỉnh này có thể được sử dụng giống như bộ điều chỉnh trước đó.

những cái:

```

bayesian_opt_tuner =
    kt.BayesianOptimization( MyClassificationHyperModel(),

    objective="val_accuracy", seed=42,
    max_trials=10, alpha=1e-4, beta=2.6, overwrite=True, directory="my_fashion_mnist",
    project_name="bayesian_opt")
bayesian_opt_tuner.search([...])

```

Điều chỉnh siêu tham số vẫn là một lĩnh vực nghiên cứu tích cực, và nhiều phương pháp khác đang được khám phá. Ví dụ, hãy xem xét DeepMind...

Bài báo xuất sắc năm 2017.²⁰ Trong đó, các tác giả đã sử dụng thuật toán tiến hóa để tối ưu hóa đồng thời một tập hợp các mô hình và các siêu tham số của chúng. Google cũng đã sử dụng phương pháp tiến hóa, không chỉ để tìm kiếm các siêu tham số mà còn để khám phá tất cả các loại kiến trúc mô hình: nó cung cấp sức mạnh cho dịch vụ AutoML của họ trên Google Vertex AI (xem Chương 19). Thuật ngữ AutoML đề cập đến bất kỳ hệ thống nào đảm nhiệm phần lớn quy trình làm việc của học máy. Các thuật toán tiến hóa thậm chí đã được sử dụng thành công để huấn luyện các mạng nơ-ron riêng lẻ, thay thế cho thuật toán Gradient Descent phổ biến!

Ví dụ, hãy xem bài đăng năm 2017. Bài viết này được thực hiện bởi Uber, trong đó các tác giả giới thiệu kỹ thuật Tiến hóa thần kinh sâu (Deep Neuroevolution) của họ.

Tuy nhiên, bất chấp tất cả những tiến bộ đáng kể và tất cả các công cụ và dịch vụ này, việc nắm được những giá trị hợp lý cho từng siêu tham số vẫn rất hữu ích để bạn có thể nhanh chóng xây dựng một nguyên mẫu và thu hẹp không gian tìm kiếm. Các phần sau đây cung cấp hướng dẫn về cách chọn số lớp ẩn và số nơ-ron trong mạng nơ-ron đa lớp (MLP) và cách chọn các giá trị tốt cho một số siêu tham số chính.

Số lượng lớp ẩn

Đối với nhiều bài toán, bạn có thể bắt đầu với một lớp ẩn duy nhất và thu được kết quả khá tốt. Về mặt lý thuyết, một mạng nơ-ron đa lớp (MLP) chỉ với một lớp ẩn có thể mô hình hóa ngay cả những hàm phức tạp nhất, miễn là nó có đủ số lượng nơ-ron.

Tuy nhiên, đối với các bài toán phức tạp, mạng nơ-ron sâu có hiệu quả tham số cao hơn nhiều so với mạng nơ-ron nông: chúng có thể mô hình hóa các hàm phức tạp bằng cách sử dụng số lượng nơ-ron ít hơn theo cấp số mũ so với mạng nơ-ron nông, cho phép chúng đạt được hiệu suất tốt hơn nhiều với cùng một lượng dữ liệu huấn luyện.

Để hiểu lý do tại sao, hãy giả sử bạn được yêu cầu vẽ một khu rừng bằng phần mềm vẽ nào đó, nhưng bạn bị cấm sao chép và dán bất cứ thứ gì. Việc đó sẽ tốn một lượng thời gian khổng lồ: bạn sẽ phải vẽ từng cây riêng lẻ, từng cành, từng lá. Nếu thay vào đó bạn có thể vẽ một chiếc lá, sao chép và dán nó để vẽ một cành cây, sau đó sao chép và dán cành cây đó để tạo thành một cái cây, và cuối cùng sao chép và dán cái cây này để tạo thành một khu rừng, bạn sẽ hoàn thành trong thời gian ngắn. Dữ liệu thực tế thường được cấu trúc theo cách phân cấp như vậy, và mạng nơ-ron sâu tự động tận dụng lợi thế của điều này.

Thực tế là: các lớp ẩn thấp hơn mô hình hóa các cấu trúc cấp thấp (ví dụ: các đoạn thẳng có hình dạng và hướng khác nhau), các lớp ẩn trung gian kết hợp các cấu trúc cấp thấp này để mô hình hóa các cấu trúc cấp trung gian (ví dụ: hình vuông, hình tròn), và các lớp ẩn cao nhất cùng lớp đầu ra kết hợp các cấu trúc trung gian này để mô hình hóa các cấu trúc cấp cao (ví dụ: các khuôn mặt).

Kiến trúc phân cấp này không chỉ giúp mạng nơ-ron sâu (DNN) hội tụ nhanh hơn đến một giải pháp tốt, mà còn cải thiện khả năng khái quát hóa của chúng đối với các tập dữ liệu mới. Ví dụ, nếu bạn đã huấn luyện một mô hình để nhận dạng khuôn mặt trong ảnh và bây giờ bạn muốn huấn luyện một mạng nơ-ron mới để nhận dạng kiểu tóc, bạn có thể bắt đầu quá trình huấn luyện bằng cách tái sử dụng các lớp thấp hơn của mạng đầu tiên. Thay vì khởi tạo ngẫu nhiên trọng số và độ lệch của một vài lớp đầu tiên của mạng nơ-ron mới, bạn có thể khởi tạo chúng bằng các giá trị trọng số và độ lệch của các lớp thấp hơn của mạng đầu tiên. Bằng cách này, mạng sẽ không phải học từ đầu tất cả các cấu trúc cấp thấp xuất hiện trong hầu hết các bức ảnh; nó chỉ cần học các cấu trúc cấp cao hơn (ví dụ: kiểu tóc). Điều này được gọi là học chuyển giao.

Tóm lại, đối với nhiều bài toán, bạn có thể bắt đầu chỉ với một hoặc hai lớp ẩn và mạng nơ-ron sẽ hoạt động tốt. Ví dụ, bạn có thể dễ dàng đạt được độ chính xác trên 97% trên tập dữ liệu MNIST chỉ với một lớp ẩn với vài trăm nơ-ron, và độ chính xác trên 98% khi sử dụng hai lớp ẩn với cùng tổng số nơ-ron, trong khoảng thời gian huấn luyện tương đương. Đối với các bài toán phức tạp hơn, bạn có thể tăng dần số lượng lớp ẩn cho đến khi bắt đầu xảy ra hiện tượng quá khớp với tập dữ liệu huấn luyện.

Các tác vụ rất phức tạp, chẳng hạn như phân loại hình ảnh lớn hoặc nhận dạng giọng nói, thường yêu cầu các mạng nơ-ron với hàng chục lớp (hoặc thậm chí hàng trăm lớp, nhưng không phải là mạng kết nối hoàn toàn, như chúng ta sẽ thấy trong [Chương 14](#)), và chúng cần một lượng dữ liệu huấn luyện khổng lồ. Bạn hiếm khi phải huấn luyện các mạng như vậy từ đầu: việc tái sử dụng các phần của một mạng nơ-ron tiên tiến đã được huấn luyện trước đó thực hiện một tác vụ tương tự phổ biến hơn nhiều. Quá trình huấn luyện sau đó sẽ nhanh hơn nhiều và yêu cầu ít dữ liệu hơn (chúng ta sẽ thảo luận điều này trong [Chương 11](#)).

Số lượng nơ-ron trên mỗi lớp ẩn

Số lượng nơ-ron trong lớp đầu vào và đầu ra được xác định bởi loại đầu vào và đầu ra mà nhiệm vụ của bạn yêu cầu. Ví dụ, nhiệm vụ MNIST yêu cầu $28 \times 28 = 784$ nơ-ron đầu vào và 10 nơ-ron đầu ra.

Đối với các lớp ẩn, trước đây người ta thường thiết kế chúng theo hình kim tự tháp, với số lượng nơ-ron giảm dần ở mỗi lớp – lý do là nhiều đặc điểm cấp thấp có thể kết hợp lại thành ít đặc điểm cấp cao hơn.

Một mạng nơ-ron điển hình cho tập dữ liệu MNIST có thể có 3 lớp ẩn, lớp đầu tiên có 300 nơ-ron, lớp thứ hai có 200 và lớp thứ ba có 100. Tuy nhiên, cách làm này phần lớn đã bị loại bỏ vì dường như việc sử dụng cùng số lượng nơ-ron trong tất cả các lớp ẩn cho hiệu quả tương đương trong hầu hết các trường hợp, hoặc thậm chí tốt hơn; hơn nữa, chỉ cần điều chỉnh một siêu tham số duy nhất, thay vì một siêu tham số cho mỗi lớp. Mặc dù vậy, tùy thuộc vào tập dữ liệu, đôi khi việc làm cho lớp ẩn đầu tiên lớn hơn các lớp khác có thể hữu ích.

Cũng giống như số lượng lớp, bạn có thể thử tăng dần số lượng nơ-ron cho đến khi mạng bắt đầu bị quá khớp. Hoặc, bạn có thể thử xây dựng một mô hình với nhiều lớp và nơ-ron hơn mức cần thiết, sau đó sử dụng dừng sớm và các kỹ thuật điều chỉnh khác để ngăn chặn tình trạng quá khớp. Vincent Vanhoucke, một nhà khoa học tại Google, đã đặt tên cho phương pháp này là "quần co giãn": thay vì lãng phí thời gian tìm kiếm chiếc quần vừa vặn hoàn hảo với kích cỡ của mình, hãy sử dụng quần co giãn rộng hơn để chúng có thể co lại vừa với kích cỡ phù hợp. Với cách tiếp cận này, bạn tránh được các lớp gây tắc nghẽn có thể làm hỏng mô hình của bạn. Thật vậy, nếu một lớp có quá ít nơ-ron, nó sẽ không có đủ khả năng biểu diễn để bảo toàn tất cả thông tin hữu ích từ đầu vào (ví dụ: một lớp có hai nơ-ron chỉ có thể xuất ra dữ liệu 2D, vì vậy nếu nó nhận dữ liệu 3D làm đầu vào, một số thông tin sẽ bị mất). Cho dù phần còn lại của mạng có lớn và mạnh đến đâu, thông tin đó sẽ không bao giờ được phục hồi.

MẸO

Nhìn chung, bạn sẽ thu được hiệu quả cao hơn bằng cách tăng số lớp thay vì tăng số lượng nơ-ron trên mỗi lớp.

Tốc độ học, kích thước lô và các siêu tham số khác

Số lớp ẩn và số nơ-ron không phải là những siêu tham số duy nhất bạn có thể điều chỉnh trong mạng nơ-ron đa lớp (MLP). Dưới đây là một số siêu tham số quan trọng nhất, cũng như các mẹo về cách thiết lập chúng:

Tốc độ học tập

Tốc độ học (learning rate) có thể nói là siêu tham số quan trọng nhất. Nói chung, tốc độ học tối ưu xấp xỉ bằng một nửa tốc độ học tối đa (tức là tốc độ học mà ở đó thuật toán huấn luyện bị phân kỳ, như chúng ta đã thấy trong [Chương 4](#)). Một cách để tìm tốc độ học tốt là huấn luyện mô hình trong vài trăm lần lặp, bắt đầu với tốc độ học rất thấp (ví dụ: 10^{-4}) và tăng dần lên đến một giá trị rất lớn (ví dụ: 10^{-6}). Điều này được thực hiện bằng cách nhân tốc độ học với một hệ số không đổi tại 0.000.

mỗi lần lặp (ví dụ: bằng $(10^{-5})^{1/500}$ lần tăng từ 10^{-5} lên 10^{-4} trong 500 lần). Nếu bạn vẽ đồ thị hàm mất mát theo tốc độ học (sử dụng thang logarit cho tốc độ học), bạn sẽ thấy nó giảm xuống lúc đầu. Nhưng sau một thời gian, tốc độ học sẽ quá lớn, vì vậy hàm mất mát sẽ tăng vọt trở lại: tốc độ học tối ưu sẽ thấp hơn một chút so với điểm mà hàm mất mát bắt đầu tăng (thường thấp hơn khoảng 10 lần so với điểm uốn). Sau đó, bạn có thể khởi tạo lại mô hình của mình và huấn luyện bình thường bằng cách sử dụng tốc độ học tốt này. Chúng ta sẽ xem xét thêm các kỹ thuật tốc độ học trong [Chương 11](#).

Trình tối ưu hóa

Việc lựa chọn một thuật toán tối ưu hóa tốt hơn thuật toán Mini-batch Gradient Descent thông thường (và điều chỉnh các siêu tham số của nó) cũng rất quan trọng. Chúng ta sẽ xem xét một số thuật toán tối ưu hóa tiên tiến trong [Chương 11](#).

Kích thước lô

Kích thước lô (batch size) có thể ảnh hưởng đáng kể đến hiệu năng và thời gian huấn luyện của mô hình. Lợi ích chính của việc sử dụng kích thước lô lớn là các bộ tăng tốc phần cứng như GPU có thể xử lý chúng một cách hiệu quả (xem [Chương 19](#)), do đó thuật toán huấn luyện sẽ thấy nhiều trường hợp hơn mỗi giây. Vì vậy, nhiều nhà nghiên cứu và chuyên gia khuyên nên sử dụng kích thước lô lớn nhất có thể chứa trong bộ nhớ RAM của GPU.

Tuy nhiên, có một vấn đề: trên thực tế, kích thước batch lớn thường dẫn đến sự bất ổn trong quá trình huấn luyện, đặc biệt là ở giai đoạn đầu huấn luyện, và mô hình thu được có thể không tổng quát hóa tốt bằng mô hình được huấn luyện với kích thước batch nhỏ. Vào tháng 4 năm 2018, Yann LeCun thậm chí đã tweet "Bạn bè không nên để bạn bè sử dụng mini-batch lớn hơn 32", trích dẫn một [bài báo năm 2018](#). ²¹ Nghiên cứu của Dominic Masters và Carlo Luschi kết luận rằng việc sử dụng các batch nhỏ (từ 2 đến 32) là tốt hơn vì chúng dẫn đến các mô hình tốt hơn trong thời gian huấn luyện ngắn hơn. Tuy nhiên, các bài báo khác lại chỉ ra hướng ngược lại; vào năm 2017, các bài báo của [Elad Hoffer và cộng sự](#) cũng như [Priya Goyal và cộng sự](#) ²² đã chỉ ra rằng có thể sử dụng kích thước batch rất lớn (lên đến 8.192) bằng nhiều kỹ thuật khác nhau như làm nóng tốc độ học (tức là bắt đầu huấn luyện với tốc độ học nhỏ, sau đó tăng dần lên, như chúng ta sẽ thấy trong [Chương 11](#)). ²³ Điều này dẫn đến thời gian huấn luyện rất ngắn, mà không có bất kỳ khoảng cách tổng quát hóa nào. Vì vậy, một chiến lược là thử sử dụng kích thước batch lớn, sử dụng phương pháp làm nóng tốc độ học, và nếu quá trình huấn luyện không ổn định hoặc hiệu suất cuối cùng không như mong đợi, thì hãy thử sử dụng kích thước batch nhỏ hơn.

Chức năng kích hoạt

Chúng ta đã thảo luận về cách chọn hàm kích hoạt ở phần trước của chương này:

nhìn chung, hàm kích hoạt ReLU sẽ là lựa chọn mặc định tốt cho tất cả các lớp ẩn. Đối với lớp đầu ra, điều đó thực sự phụ thuộc vào nhiệm vụ của bạn.

Số lần lặp

Trong hầu hết các trường hợp, số lần lặp huấn luyện thực tế không cần phải điều chỉnh: chỉ cần sử dụng phương pháp dừng sớm là được.

MẸO

Tốc độ học tối ưu phụ thuộc vào các siêu tham số khác-đặc biệt là kích thước lô-vì vậy nếu bạn thay đổi bất kỳ siêu tham số nào, hãy nhớ cập nhật cả tốc độ học.

Để biết thêm các phương pháp tối ưu hóa tham số siêu tham số của mạng nơ-ron, hãy tham khảo [bài báo xuất sắc năm 2018](#). Tác giả: ²⁴ Leslie Smith.

Như vậy là phần giới thiệu về mạng nơ-ron nhân tạo và cách triển khai chúng bằng Keras đã kết thúc. Trong các chương tiếp theo, chúng ta sẽ thảo luận về các kỹ thuật huấn luyện mạng nơ-ron rất sâu. Chúng ta cũng sẽ tìm hiểu cách tùy chỉnh mô hình bằng API cấp thấp của TensorFlow và cách tải và tiền xử lý dữ liệu hiệu quả bằng API Dữ liệu. Và chúng ta sẽ đi sâu vào các kiến trúc mạng nơ-ron phổ biến khác: mạng nơ-ron tích chập cho xử lý ảnh, mạng nơ-ron hồi quy và transformer cho dữ liệu tuần tự và văn bản, autoencoder cho học biểu diễn, và mạng nơ-ron tạo sinh.

Mạng đối kháng được sử dụng để mô hình hóa và tạo dữ liệu. ²⁵

Bài tập

1. **Sân chơi TensorFlow** Đây là một trình mô phỏng mạng nơ-ron tiện dụng được xây dựng bởi nhóm TensorFlow. Trong bài tập này, bạn sẽ huấn luyện một số bộ phân loại nhị phân chỉ với vài cú nhấp chuột, và điều chỉnh kiến trúc của mô hình cũng như các siêu tham số của nó để hiểu rõ hơn về cách hoạt động của mạng nơ-ron và chức năng của các siêu tham số. Hãy dành chút thời gian để khám phá những điều sau:

- a. Các mẫu được học bởi mạng nơ-ron. Hãy thử huấn luyện mô hình mặc định. Bạn có thể chạy mạng nơ-ron bằng cách nhấp vào nút Chạy (góc trên bên trái). Hãy chú ý cách nó nhanh chóng tìm ra một giải pháp tốt cho nhiệm vụ phân loại. Các nơ-ron trong lớp ẩn đầu tiên đã học được các mẫu đơn giản, trong khi các nơ-ron trong lớp ẩn thứ hai đã học cách kết hợp các mẫu đơn giản của lớp ẩn đầu tiên thành các mẫu phức tạp hơn.

Các họa tiết phức tạp. Nói chung, càng nhiều lớp thì họa tiết càng phức tạp.

- b. Hàm kích hoạt. Hãy thử thay thế hàm kích hoạt tanh bằng hàm kích hoạt ReLU và huấn luyện lại mạng.
Hãy lưu ý rằng nó tìm ra lời giải thậm chí còn nhanh hơn, nhưng lần này các ranh giới là tuyến tính. Điều này là do hình dạng của hàm ReLU.
- c. Nguy cơ cực tiểu cục bộ. Điều chỉnh kiến trúc mạng để chỉ có một lớp ẩn với ba nơ-ron. Huấn luyện nhiều lần (để đặt lại trọng số mạng, hãy nhấp vào nút Đặt lại bên cạnh nút Phát). Lưu ý rằng thời gian huấn luyện thay đổi rất nhiều, và đôi khi nó thậm chí còn bị kẹt ở cực tiểu cục bộ.
- d. Điều gì xảy ra khi mạng nơ-ron quá nhỏ? Hãy loại bỏ một nơ-ron.
Chỉ giữ lại hai nơ-ron. Lưu ý rằng mạng nơ-ron hiện không thể tìm ra giải pháp tốt, ngay cả khi bạn thử nhiều lần. Mô hình có quá ít tham số và luôn luôn không phù hợp với tập dữ liệu huấn luyện.
- e. Điều gì xảy ra khi mạng nơ-ron đủ lớn? Hãy đặt số lượng nơ-ron là tám và huấn luyện mạng nhiều lần. Bạn sẽ thấy rằng mạng lúc này hoạt động nhanh ổn định và không bao giờ bị kẹt. Điều này làm nổi bật một phát hiện quan trọng trong lý thuyết mạng nơ-ron: các mạng nơ-ron lớn hiếm khi bị kẹt ở các cực tiểu cục bộ, và ngay cả khi bị kẹt, các cực tiểu cục bộ này thường gần tốt bằng cực đại toàn cục. Tuy nhiên, chúng vẫn có thể bị kẹt ở các vùng ổn định kéo dài trong một thời gian dài.
- f. Nguy cơ mất độ dốc trong mạng nơ-ron sâu. Chọn tập dữ liệu hình xoắn ốc (tập dữ liệu phía dưới bên phải trong mục "DATA") và thay đổi kiến trúc mạng thành bốn lớp ẩn với tám nơ-ron mỗi lớp. Lưu ý rằng quá trình huấn luyện mất nhiều thời gian hơn và thường bị kẹt ở trạng thái ổn định trong thời gian dài. Cũng lưu ý rằng các nơ-ron trong các lớp cao nhất (bên phải) có xu hướng phát triển.

nhanh hơn các nơ-ron ở các lớp thấp nhất (bên trái). Vấn đề này, được gọi là vấn đề "độ dốc biến mất", có thể được giảm nhẹ bằng cách khởi tạo trọng số tốt hơn và các kỹ thuật khác, các thuật toán tối ưu hóa tốt hơn (như AdaGrad hoặc Adam), hoặc Chuẩn hóa theo lô (được thảo luận trong [Chương 11](#)).

g. Tiến xa hơn. Dành khoảng một giờ để thử nghiệm với các tùy chọn khác.

Tìm hiểu các tham số và nắm bắt chức năng của chúng để xây dựng sự hiểu biết trực quan về mạng nơ-ron.

2. Vẽ một mạng nơ-ron nhân tạo (ANN) sử dụng các nơ-ron nhân tạo ban đầu (như trong [Hình 10-3](#)) để tính toán $A \oplus B$ (trong đó $\oplus$ biểu thị phép toán XOR). Gợi ý: $A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$.

3. Tại sao việc sử dụng bộ phân loại Hồi quy Logistic thường được ưa chuộng hơn? Thay vì sử dụng Perceptron cổ điển (tức là một lớp đơn các đơn vị logic ngưỡng được huấn luyện bằng thuật toán huấn luyện Perceptron)? Làm thế nào để điều chỉnh Perceptron để nó tương đương với bộ phân loại Hồi quy Logistic?

4. Tại sao hàm kích hoạt sigmoid lại là một thành phần quan trọng trong việc huấn luyện các mạng MLP đầu tiên?

5. Hãy nêu tên ba hàm kích hoạt phổ biến. Bạn có thể vẽ chúng không?

6. Giả sử bạn có một mạng MLP gồm một lớp đầu vào với 10

Cấu trúc gồm các nơ-ron truyền qua, tiếp theo là một lớp ẩn với 50 nơ-ron nhân tạo, và cuối cùng là một lớp đầu ra với 3 nơ-ron nhân tạo. Tất cả các nơ-ron nhân tạo đều sử dụng hàm kích hoạt ReLU.

- Ma trận đầu vào X có hình dạng như thế nào ?
- Hình dạng của ma trận trọng số W và vectơ độ lệch b của lớp ẩn là gì ?
 h
- Hình dạng của ma trận trọng số W và vectơ độ lệch b của lớp đầu ra là gì ?
 o
- Hình dạng của ma trận đầu ra Y của mạng là gì ?

- Viết phương trình tính ma trận đầu ra Y của mạng như một hàm của X , W , b , σ , và b_o .

7. Bạn cần bao nhiêu nơ-ron ở lớp đầu ra nếu bạn muốn...

Phân loại email thành thư rác hay thư hợp lệ? Bạn nên kích hoạt chức năng nào?

Sử dụng ở lớp đầu ra? Nếu thay vào đó bạn muốn xử lý tập dữ liệu MNIST thì sao?

Bạn cần bao nhiêu nơ-ron ở lớp đầu ra và cần kích hoạt loại nào?

Bạn nên sử dụng hàm nào? Còn việc kết nối mạng thì sao?

Dự đoán giá nhà ở, như trong **Chương 2**?

8. Lan truyền ngược là gì và nó hoạt động như thế nào? Sự khác biệt giữa nó và lan truyền ngược là gì?

Giữa phương pháp lan truyền ngược và phương pháp tự sai phân ngược?

9. Bạn có thể liệt kê tất cả các siêu tham số mà bạn có thể điều chỉnh trong một mạng nơ-ron đa lớp cơ bản không? Nếu

Mạng MLP bị quá khớp với dữ liệu huấn luyện, làm thế nào để điều chỉnh những điều này?

Các siêu tham số nào có thể được sử dụng để giải quyết vấn đề?

10. Huấn luyện một mạng MLP sâu trên tập dữ liệu MNIST (bạn có thể tải nó bằng cách sử dụng

`tf.keras.datasets.mnist.load_data()`). Hãy xem bạn có thể làm được không

Đạt độ chính xác trên 98% bằng cách tinh chỉnh thủ công các siêu tham số. Hãy thử xem!

Tìm kiếm tốc độ học tối ưu bằng cách sử dụng phương pháp được trình bày

trong chương này (tức là bằng cách tăng tốc độ học theo cấp số nhân,

(Vẽ biểu đồ tổn thất và tìm điểm mà tổn thất tăng vọt). Tiếp theo,

Hãy thử tinh chỉnh các siêu tham số bằng Keras Tuner với tất cả các tùy chọn (bells) và

còi báo hiệu – lưu điểm kiểm tra, sử dụng chức năng dừng sớm và lập kế hoạch học tập

Vẽ đường cong bằng TensorBoard.

Đáp án cho các bài tập này có ở cuối chương này.

notebook, tại <https://homl.info/colab3>.

1. Bạn có thể đạt được những điều tốt nhất từ cả hai phía bằng cách cởi mở đón nhận những nguồn cảm hứng sinh học mà không cần phải...
e ngại việc tạo ra các mô hình không thực tế về mặt sinh học, miễn là chúng hoạt động tốt.

2 Warren S. McCulloch và Walter Pitts, "Một phép tính logic về các ý niệm nội tại trong
Hoạt động thần kinh," Bản tin Sinh học Toán học 5, số 4 (1943): 115-113.

3. Chúng không thực sự dính liền với nhau, chỉ ở rất gần nhau đến mức có thể trao đổi chất hóa học rất nhanh chóng.
tín hiệu.

4. Hình ảnh của Bruce Blaus (Creative Commons 3.0). Sao chép từ <https://en.wikipedia.org/wiki/Neuron>.
 5. Trong bối cảnh Học máy, cụm từ "mạng nơ-ron" thường dùng để chỉ mạng nơ-ron nhân tạo (ANN). Không phải BNN.
 6. Bản vẽ cấu trúc lớp vỏ não của S. Ramon y Cajal (thuộc phạm vi công cộng). Được sao chép từ... https://en.wikipedia.org/wiki/Cerebral_cortex.
 7. Lưu ý rằng giải pháp này không phải là duy nhất: khi các điểm dữ liệu có thể phân tách tuyến tính, sẽ có một Vô số siêu mặt phẳng có thể phân tách chúng.
 8. Ví dụ, khi đầu vào là (0, 1), nơ-ron phía dưới bên trái tính toán $0 \times 1 + 1 \times 1 - 3/2 = -1/2$, là số âm nên nó xuất ra 0. Nơ-ron phía dưới bên phải tính toán $0 \times 1 + 1 \times 1 - 1/2 = 1/2$, là số dương nên nó xuất ra 1. Nơ-ron đầu ra nhận đầu ra của hai nơ-ron đầu tiên làm đầu vào, vì vậy nó tính toán $0 \times (-1) + 1 \times 1 - 1/2 = 1/2$, là số dương nên nó xuất ra 1.
 9. Vào những năm 1990, một mạng nơ-ron nhân tạo (ANN) có nhiều hơn hai lớp ẩn được coi là "sâu". Ngày nay, việc thấy các ANN có hàng chục, thậm chí hàng trăm lớp là điều phổ biến, vì vậy định nghĩa về "sâu" khá mơ hồ.
 - 10 David Rumelhart và cộng sự. "Học các biểu diễn nội bộ bằng cách lan truyền lỗi," (Bảo vệ) Báo cáo kỹ thuật của Trung tâm Thông tin Kỹ thuật, tháng 9 năm 1985).
 11. Các nơ-ron sinh học dường như thực hiện một hàm kích hoạt gần giống hình chữ S (sigmoid), vì vậy các nhà nghiên cứu đã gắn bó với hàm sigmoid trong một thời gian rất dài. Nhưng hóa ra hàm ReLU thường hoạt động tốt hơn trong mạng nơ-ron nhân tạo (ANN). Đây là một trong những trường hợp mà sự tương đồng sinh học có lẽ đã gây hiểu lầm.
 12. Dự án ONEIROs (Hệ điều hành robot thông minh điện tử thần kinh mở).
- API của **PyTorch** khá giống với Keras, vì vậy một khi bạn đã quen với Keras, việc chuyển sang PyTorch sẽ không khó nếu bạn muốn. Sự phổ biến của PyTorch đã tăng lên theo cấp số nhân vào năm 2018, phần lớn nhờ vào sự đơn giản và tài liệu tuyệt vời của nó, những điểm mạnh mà TensorFlow 1.x không có được vào thời điểm đó. Tuy nhiên, TensorFlow 2 cũng đơn giản như PyTorch, một phần vì nó đã áp dụng Keras làm API cấp cao chính thức, và cũng vì các nhà phát triển đã đơn giản hóa và làm sạch đáng kể phần còn lại của API. Tài liệu cũng đã được tổ chức lại hoàn toàn, và giờ đây việc tìm kiếm những gì bạn cần dễ dàng hơn nhiều. Tương tự, những điểm yếu chính của PyTorch (ví dụ: tính di động hạn chế và không có phân tích đồ thị tính toán) đã được giải quyết phần lớn trong PyTorch 1.0. Cạnh tranh lành mạnh dường như có lợi cho tất cả mọi người.
14. Bạn cũng có thể sử dụng `tf.keras.utils.plot_model()` để tạo hình ảnh của mô hình của mình. người mẫu.
 - 15 Heng-Tze Cheng và cộng sự, "Học sâu và toàn diện cho hệ thống đề xuất," Kỷ yếu Hội thảo đầu tiên về Học sâu cho hệ thống đề xuất (2016): 7-10.
 16. Đường dẫn ngắn cũng có thể được sử dụng để cung cấp các tính năng được thiết kế thủ công cho mạng nơ-ron. mạng.
 - 17 mô hình Keras có thuộc tính đầu ra, vì vậy chúng ta không thể sử dụng tên đó cho đầu ra chính. lớp, đó là lý do tại sao chúng tôi đổi tên nó thành `main_output`.

18. Hiện tại đây là định dạng mặc định, nhưng nhóm Keras đang nghiên cứu một định dạng mới có thể trở thành định dạng mặc định trong các phiên bản sắp tới, vì vậy tôi muốn thiết lập định dạng một cách rõ ràng để đảm bảo tính tương thích trong tương lai.
19. Hyperband thực chất phức tạp hơn một chút so với phương pháp chia đôi liên tiếp, xem [bài báo](#). bởi Lisha Li và cộng sự, "Hyperband: Một phương pháp tiếp cận mới dựa trên thuật toán Bandit để tối ưu hóa siêu tham số," Tạp chí nghiên cứu máy học 18 (tháng 4 năm 2018): 1-52.
- 20 Max Jaderberg và cộng sự, "Huấn luyện mạng nơ-ron dựa trên quần thể," bản thảo arXiv arXiv:1711.09846 (2017).
- 21 Dominic Masters và Carlo Luschi, "Xem xét lại việc huấn luyện theo lô nhỏ cho mạng nơ-ron sâu" Mạng lưới," bản in trước arXiv arXiv:1804.07612 (2018).
- 22 Elad Hoffer và cộng sự, "Huấn luyện lâu hơn, khái quát hóa tốt hơn: Thu hẹp khoảng cách khái quát hóa trong huấn luyện hàng loạt lớn của mạng nơ-ron," Kỷ yếu Hội nghị quốc tế lần thứ 31 về Hệ thống xử lý thông tin thần kinh (2017): 1729-1739.
- 23 Priya Goyal và cộng sự, "SGD xử lý hàng loạt lớn, chính xác: Huấn luyện ImageNet trong 1 giờ," arXiv Bản thảo trước khi xuất bản arXiv:1706.02677 (2017).
- 24 Leslie N. Smith, "Một cách tiếp cận có kỷ luật đối với các siêu tham số của mạng nơ-ron: Phần 1—"Tốc độ học, kích thước lô, động lượng và sự suy giảm trọng số," bản thảo arXiv arXiv:1803.09820 (2018).
- 25 Một vài kiến trúc ANN bổ sung được trình bày trong sổ tay trực tuyến tại <https://homl.info/extra-anns>.

Chương 11. Huấn luyện mạng nơ-ron sâu

Với sách điện tử phát hành sớm, bạn sẽ nhận được sách ở dạng sơ khai nhất—nội dung thô và chưa chỉnh sửa của tác giả khi họ viết—nhờ đó bạn có thể tận dụng những công nghệ này rất lâu trước khi các tựa sách được phát hành chính thức.

Đây sẽ là chương thứ 11 của cuốn sách cuối cùng. Kho lưu trữ GitHub là <https://github.com/ageron/handson-ml3>.

Nếu bạn có ý kiến đóng góp về cách chúng tôi có thể cải thiện nội dung và/hoặc ví dụ trong cuốn sách này, hoặc nếu bạn nhận thấy thiếu sót nào trong chương này, vui lòng liên hệ với biên tập viên theo địa chỉ mcronin@oreilly.com.

Trong **Chương 10**, bạn đã xây dựng, huấn luyện và tinh chỉnh mạng nơ-ron nhân tạo đầu tiên của mình. Nhưng đó chỉ là những mạng nông, với một vài lớp ẩn. Điều gì sẽ xảy ra nếu bạn cần giải quyết một vấn đề phức tạp, chẳng hạn như phát hiện hàng trăm loại đối tượng trong hình ảnh độ phân giải cao? Bạn có thể cần huấn luyện một mạng nơ-ron nhân tạo sâu hơn nhiều, có thể với 10 lớp hoặc nhiều hơn nữa, mỗi lớp chứa hàng trăm nơ-ron, được liên kết bởi hàng trăm nghìn kết nối. Huấn luyện một mạng nơ-ron sâu không phải là chuyện dễ dàng. Dưới đây là một số vấn đề bạn có thể gặp phải:

- Bạn có thể gặp phải vấn đề khó giải quyết là hiện tượng suy giảm độ dốc (vanishing gradients) hoặc vấn đề liên quan là hiện tượng bùng nổ độ dốc (exploding gradients). Điều này xảy ra khi độ dốc ngày càng nhỏ đi hoặc ngày càng lớn đi khi truyền ngược trở lại trong mạng nơ-ron sâu (DNN) trong quá trình huấn luyện. Cả hai vấn đề này đều khiến việc huấn luyện các lớp thấp hơn trở nên rất khó khăn.
- Bạn có thể không có đủ dữ liệu huấn luyện cho một mạng lưới lớn như vậy, hoặc việc gán nhãn dữ liệu có thể quá tốn kém.

- Quá trình huấn luyện có thể diễn ra cực kỳ chậm.
- Một mô hình với hàng triệu tham số sẽ có nguy cơ quá khớp với tập dữ liệu huấn luyện, đặc biệt nếu không có đủ mẫu huấn luyện hoặc nếu chúng quá nhiều.

Trong chương này, chúng ta sẽ xem xét từng vấn đề này và trình bày các kỹ thuật để giải quyết chúng. Chúng ta sẽ bắt đầu bằng việc tìm hiểu các vấn đề về độ dốc biến mất và bùng nổ, cùng một số giải pháp phổ biến nhất của chúng.

Tiếp theo, chúng ta sẽ xem xét học chuyển giao và huấn luyện trước không giám sát, những phương pháp có thể giúp bạn giải quyết các nhiệm vụ phức tạp ngay cả khi bạn có ít dữ liệu được gắn nhãn. Tiếp theo, chúng ta sẽ thảo luận về các thuật toán tối ưu hóa khác nhau có thể tăng tốc quá trình huấn luyện các mô hình lớn một cách đáng kể. Cuối cùng, chúng ta sẽ đề cập đến một vài kỹ thuật điều chỉnh phổ biến cho các mạng nơ-ron lớn.

Với những công cụ này, bạn sẽ có thể huấn luyện các mạng nơ-ron rất sâu. Chào mừng đến với Học sâu!

Sự biến mất/bùng nổ của các dải màu

Vấn đề

Như chúng ta đã thảo luận trong [Chương 10](#), giai đoạn thứ hai của thuật toán lan truyền ngược hoạt động bằng cách đi từ lớp đầu ra đến lớp đầu vào, lan truyền độ dốc lỗi dọc theo đường đi. Sau khi thuật toán đã tính toán độ dốc của hàm chi phí đối với mỗi tham số trong mạng, nó sử dụng các độ dốc này để cập nhật từng tham số bằng bước Giảm độ dốc (Gradient Descent).

Thật không may, độ dốc thường giảm dần khi thuật toán tiến xuống các lớp thấp hơn. Kết quả là, quá trình cập nhật Gradient Descent hầu như không làm thay đổi trọng số kết nối của các lớp thấp hơn, và quá trình huấn luyện không bao giờ hội tụ đến một giải pháp tốt. Đây được gọi là vấn đề độ dốc biến mất. Trong một số trường hợp, điều ngược lại có thể xảy ra: độ dốc có thể tăng lên cho đến khi các lớp nhận được các cập nhật trọng số cực lớn và thuật toán phân kỳ. Đây là vấn đề độ dốc bùng nổ,

Các bề mặt này thường xuất hiện nhất trong mạng nơ-ron hồi quy (xem [Chương 15](#)). Nói chung, mạng nơ-ron sâu gặp phải vấn đề về độ dốc không ổn định; các lớp khác nhau có thể học với tốc độ rất khác nhau.

Hiện tượng không mong muốn này đã được quan sát thực nghiệm từ lâu, và đó là một trong những lý do khiến mạng nơ-ron sâu (DNN) hầu hết bị bỏ rơi vào đầu những năm 2000. Nguyên nhân gây ra sự không ổn định của gradient khi huấn luyện DNN vẫn chưa rõ ràng, nhưng một số thông tin đã được hé lộ trong một [bài báo năm 2010](#). Bài nghiên cứu của Xavier Glorot và Yoshua Bengio cho thấy một vài nguyên nhân đáng nghi ngờ, bao gồm sự kết hợp giữa hàm kích hoạt sigmoid (logistic) phổ biến và kỹ thuật khởi tạo trọng số phổ biến nhất vào thời điểm đó (tức là phân phối chuẩn với giá trị trung bình là 0 và độ lệch chuẩn là 1). Tóm lại, họ đã chỉ ra rằng với hàm kích hoạt và sơ đồ khởi tạo này, phương sai của đầu ra của mỗi lớp lớn hơn nhiều so với phương sai của đầu vào. Càng đi sâu vào mạng, phương sai càng tăng sau mỗi lớp cho đến khi hàm kích hoạt bão hòa ở các lớp trên cùng. Sự bão hòa này càng trở nên tồi tệ hơn bởi thực tế là hàm sigmoid có giá trị trung bình là 0,5 chứ không phải 0 (hàm hyperbolic tangent có giá trị trung bình là 0 và hoạt động tốt hơn một chút so với hàm sigmoid trong mạng nơ-ron sâu).

Khi xem xét hàm kích hoạt sigmoid (xem [Hình 11-1](#)), bạn có thể thấy rằng khi đầu vào trở nên lớn (âm hoặc dương), hàm này bão hòa ở 0 hoặc 1, với đạo hàm cực kỳ gần bằng 0 (tức là đường cong phẳng ở cả hai cực). Do đó, khi quá trình lan truyền ngược diễn ra, nó hầu như không có độ dốc nào để lan truyền ngược trở lại qua mạng; và độ dốc ít ỏi còn lại tiếp tục bị pha loãng khi quá trình lan truyền ngược tiến xuống các lớp trên cùng, vì vậy thực sự không còn gì cho các lớp thấp hơn.

